Lundi 12/10 — Algorithmes et projet Scratch

Kevin Bardoux

lundi 12 octobre 2026

Table des matières

Objectifs 2
Matin — CS50 Week 0 — algorithmes & Scratch (exercices) 2
Rappel (5 min) 2
Apport 1 — Du débranché au pseudo-code (15 min) 2
Pseudo-code — attention et vérification 4
Exercice 1 — « Couper en deux » en pseudo-code (15 min) 4
Exercice 2 — Deviner un nombre (20 min) 4
Apport 2 — Les variables dans Scratch (15 min) 4
Variables Scratch — attention et vérification 17
Exercices 3 à 5 — Scratch (45 min) 17
Si le temps le permet — « Deux pages à la fois » 18
À retenir — ce matin 18
Débrief 19
Après-midi — TP — « Problem set 0 : projet Scratch » 19
Apport 3 — Tout est 0 et 1 (15 min) 19
Binaire — attention et vérification 20
Apport 4 — Dans l'ordinateur (5 min) 20
Dans l'ordinateur — attention et vérification 21
Apport 5 — Les messages entre sprites (10 min) 22
Messages — attention et vérification 39
Le projet — consignes (2 h 10) 39
Pas à pas — enregistrer votre projet sur l'ordinateur 39
Cahier des charges 41
Si le temps le permet 42

Pour aller plus loin 42
À retenir — cet après-midi 42
Sources 43

> Note
>
> **Compétences visées** : résolution de problème · communiquer
> **Livrable** : votre projet Scratch commencé (lien ou fichier `.sb3`).

Objectifs

- Écrire un algorithme en pseudo-code.
- Comparer des algorithmes de recherche en comptant les pages.
- Utiliser une variable et un message dans Scratch.
- Démarrer un projet Scratch à partir d'un cahier des charges.

Comment se passe la journée. Chaque notion suit le même ordre : pourquoi, définition, exemple, pas à pas, erreur fréquente, vérification. Un encadré « À retenir » termine chaque demi-journée.

Matin — CS50 Week 0 — algorithmes & Scratch (exercices)

Rappel (5 min)

1. Qu'est-ce qu'un algorithme ?
2. Pourquoi « couper en deux » exige-t-il une liste triée ?
3. Quel bloc Scratch démarre un script ?

Trois questions courtes, sans regarder vos notes : la veille, la semaine passée, puis plus loin. Répondez sur une feuille ; la correction suit, à l'oral.

Apport 1 — Du débranché au pseudo-code (15 min)

- **Pourquoi** : écrire un algorithme avant de le programmer.
- **Pseudo-code** : du français structuré, une instruction par ligne.

Pourquoi. Vendredi, vous avez décrit à voix haute comment chercher un nom dans l'annuaire. Pour qu'un autre lecteur (puis un

ordinateur) suive la méthode sans hésiter, il faut l'écrire de façon précise.

Définition. Le pseudo-code est un algorithme écrit en français structuré, avant le vrai code : une instruction par ligne, avec les mêmes mots qu'en semaine 1 (←, `SI … ALORS`, `TANT QUE`, `POUR`), et un décalage vers la droite pour ce qui est « à l'intérieur » d'un `SI` ou d'un `TANT QUE`. Il n'a pas de règle officielle : l'important est qu'un autre lecteur puisse le suivre. Les exemples de l'annuaire viennent du cours CS50 (CS50 2026a).

Exemple. La méthode « page par page » :

```
ouvrir l'annuaire à la première page
TANT QUE le nom n'est pas sur la page
    SI c'est la dernière page ALORS
        AFFICHER "absent"
        ARRÊTER
    FIN SI
    tourner une page
FIN TANT QUE
AFFICHER "trouvé"
```

Pas à pas. Le formateur « joue » l'algorithme avec un vrai livre ; vous suivez sur votre feuille, ligne par ligne.

1. Ligne 1 : ouvrir le livre à la première page.
2. Ligne 2 : le nom est-il sur la page ? Non : on entre dans la boucle.
3. Ligne 3 : est-ce la dernière page ? Non : on saute jusqu'à `FIN SI`.
4. Ligne 7 : tourner une page, puis revenir à la ligne 2.
5. Quand le nom est sur la page, la condition de la ligne 2 est fausse : on sort de la boucle et on affiche `trouvé`.

Pseudo-code — attention et vérification

> ⚠ Attention
>
> Un algorithme qui oublie le cas « absent » ne s'arrête jamais quand le nom n'existe pas. Prévoyez toujours l'échec.

- **Vérifiez** : dans l'exemple, quelle ligne gère le nom absent ?

Exercice 1 — « Couper en deux » en pseudo-code (15 min)

1. Ensemble (5 min) : la première ligne et le `TANT QUE`.
2. Seuls (7 min) : écrivez la suite de « couper en deux ».
3. Avec un voisin (3 min) : l'un lit, l'autre joue avec un livre.
4. **Réussi si** : trois issues (trouvé, gauche, droite) et le cas « absent ».

> 💡 Indice
>
> Gardez une trace des pages qui restent possibles. À chaque tour : ouvrir la page du milieu, comparer le nom, puis garder la moitié de gauche ou de droite. Quand il ne reste plus de page, le nom est absent.

Exercice 2 — Deviner un nombre (20 min)

1. Par deux : un **meneur**, un **joueur** (règles ci-dessous).
2. Trois parties, puis échangez les rôles.
3. Seuls : la stratégie en 7 essais au plus, en pseudo-code.
4. **Réussi si** : elle trouve 73 en 7 essais au plus.

Le **meneur** pense à un nombre entre 1 et 100. Le **joueur** propose un nombre ; le meneur répond seulement « plus grand », « plus petit » ou « trouvé ». Cherchez la stratégie qui trouve toujours en **7 essais au plus**, puis écrivez-la en pseudo-code.

Apport 2 — Les variables dans Scratch (15 min)

- **Pourquoi** : retenir une valeur qui change, comme un score.
- **Variable** : un nom qui désigne une valeur.

Pourquoi. Un jeu compte des points, une histoire retient le prénom du joueur : le programme doit garder une valeur et la modifier.

Définition. Une variable est un nom qui désigne une valeur. Dans Scratch : catégorie **Variables**, bouton **Créer une variable** ; « mettre … à … » donne une valeur ; « ajouter … à … » l'augmente (ou la diminue, avec un nombre négatif) ; « demander … et attendre », puis « réponse », lit ce que l'on tape. Les noms des blocs suivent la traduction française officielle de Scratch (Scratch Foundation, s. d.-b).

Une image, avec sa limite. On peut imaginer une **étiquette** portant le nom, collée sur une valeur : « mettre score à 0 » recolle l'étiquette sur 0. Demain, en JavaScript, c'est la même image. Sa limite apparaîtra en semaine 6, quand deux étiquettes pourront désigner le même objet.

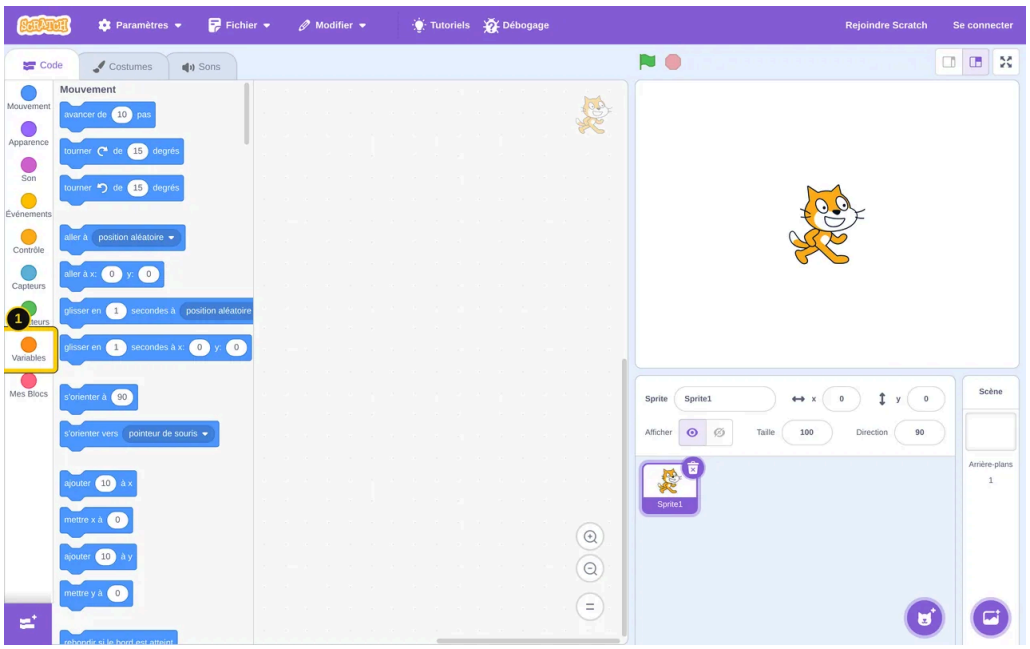
Exemple.

```
quand ⚑ est cliqué
mettre score à 0
quand ce sprite est cliqué
ajouter 1 à score
```

Pas à pas. Le formateur fait chaque étape ; vous la refaites aussitôt dans Scratch. Sur chaque capture, l'endroit à cliquer est entouré d'un cadre jaune, avec le numéro de l'étape ; un zoom sur la zone utile suit chaque capture. Sur téléphone, touchez une capture pour l'agrandir.

Ouvrez l'éditeur de Scratch : https://scratch.mit.edu/projects/editor/. Ce site mesure sa fréquentation.

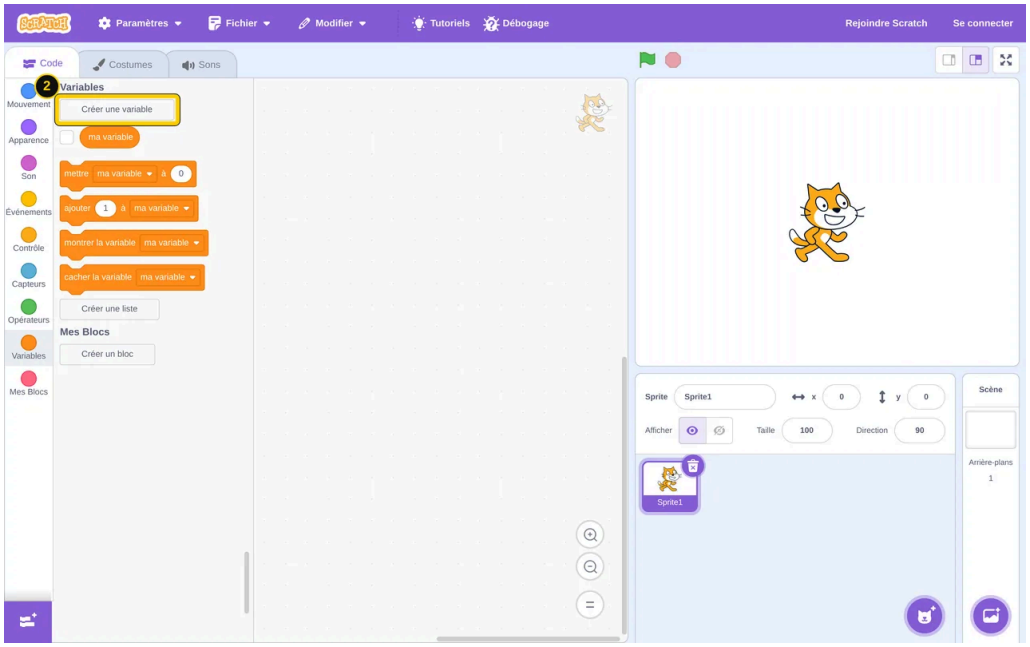
Étape 1. Dans la colonne de gauche, cliquez sur la catégorie **Variables** (rond orange).





Vous devez voir : la palette affiche les blocs orange des variables, sous le bouton « Créer une variable ».

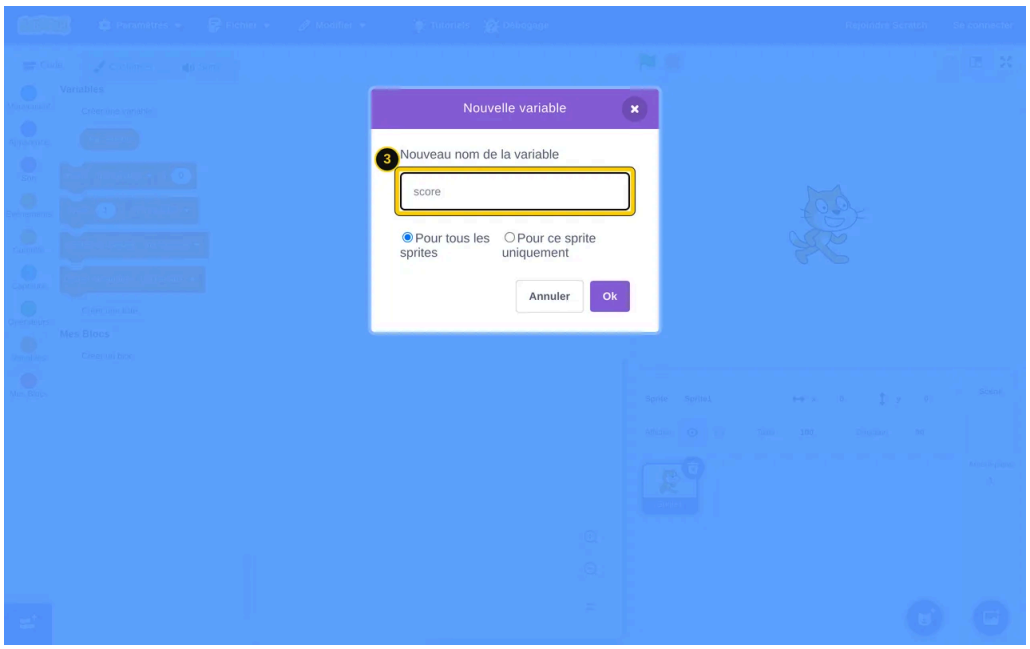
Étape 2. Cliquez sur **Créer une variable**.

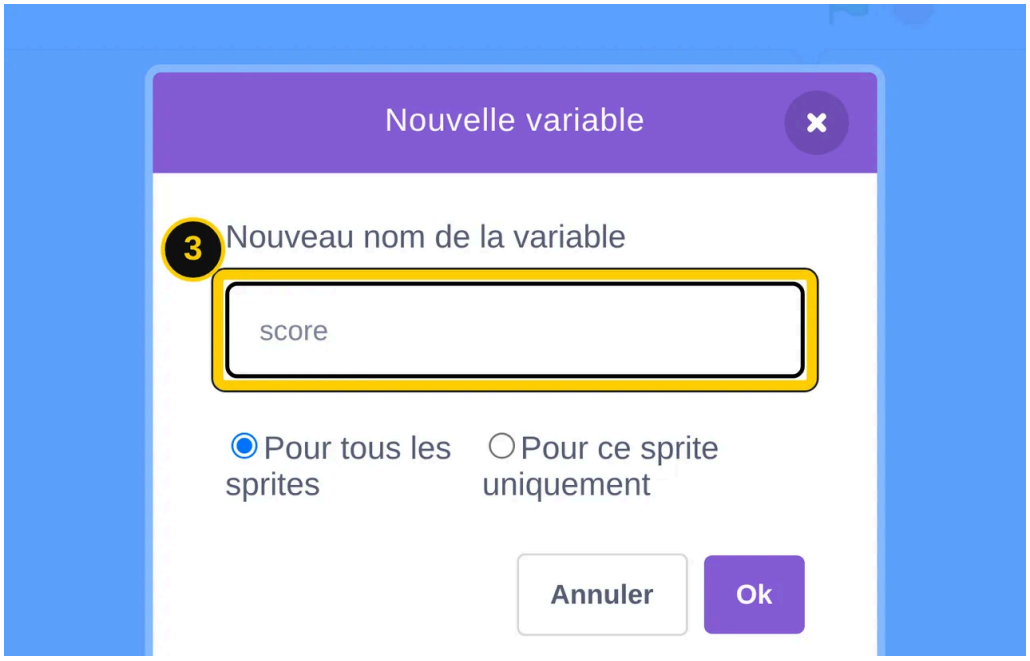




Vous devez voir : la fenêtre « Nouvelle variable ».

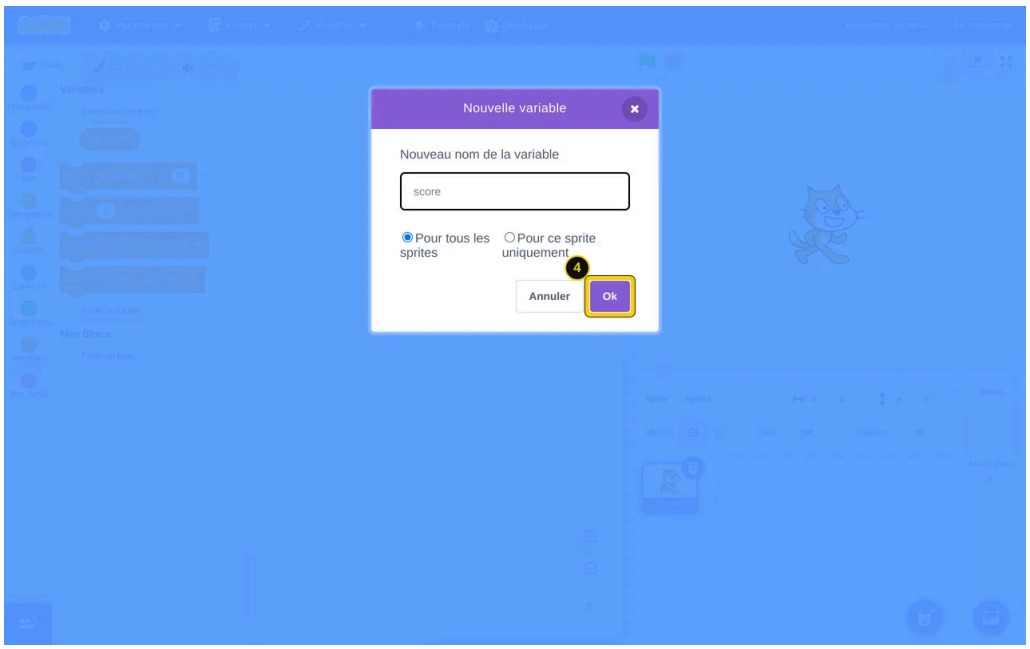
Étape 3. Dans le champ « Nouveau nom de la variable », tapez `score`.





Vous devez voir : `score` dans le champ. En dessous, « Pour tous les sprites » est déjà coché : gardez ce choix.

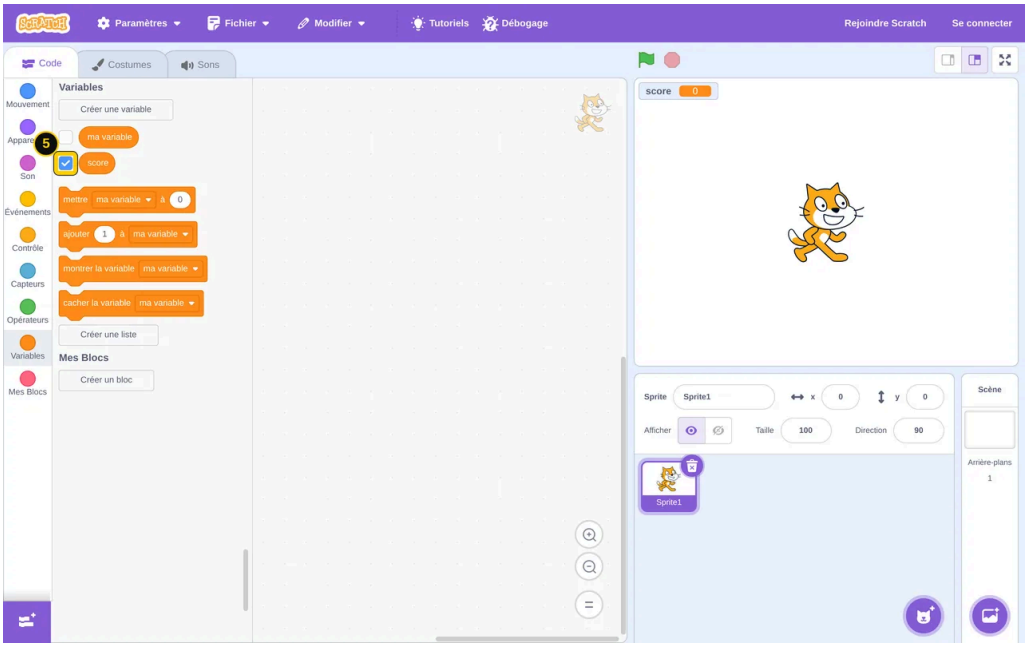
Étape 4. Cliquez sur **Ok**.





Vous devez voir : `score` dans la palette, sous « ma variable ».

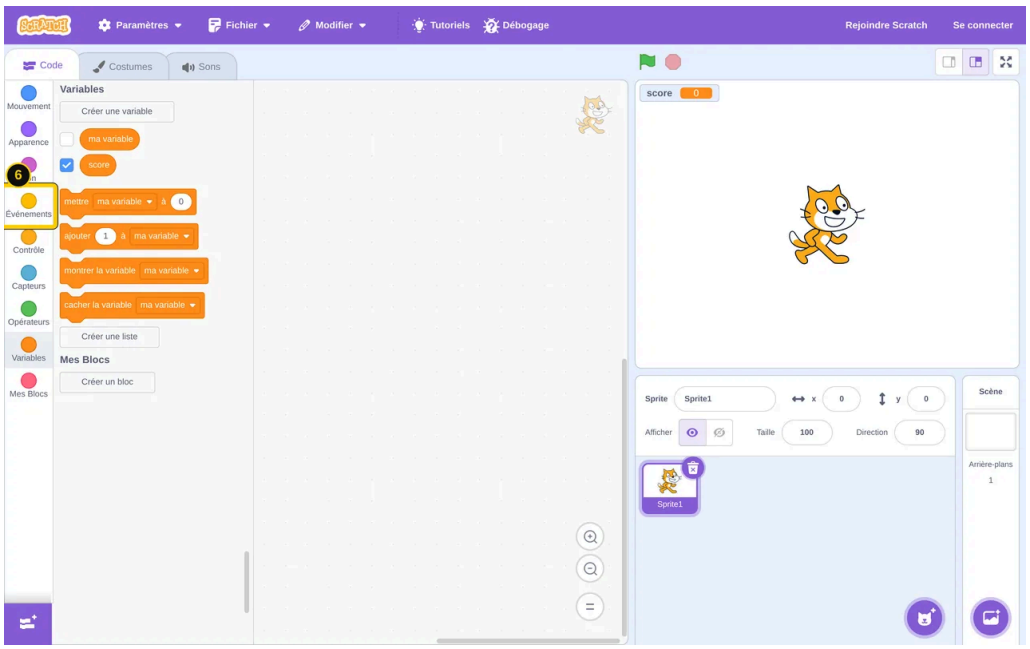
Étape 5. Regardez la case à gauche de `score` : Scratch l'a déjà cochée. Si elle ne l'est pas, cliquez dessus.





Vous devez voir : en haut à gauche de la scène, l'affichage `score 0`.

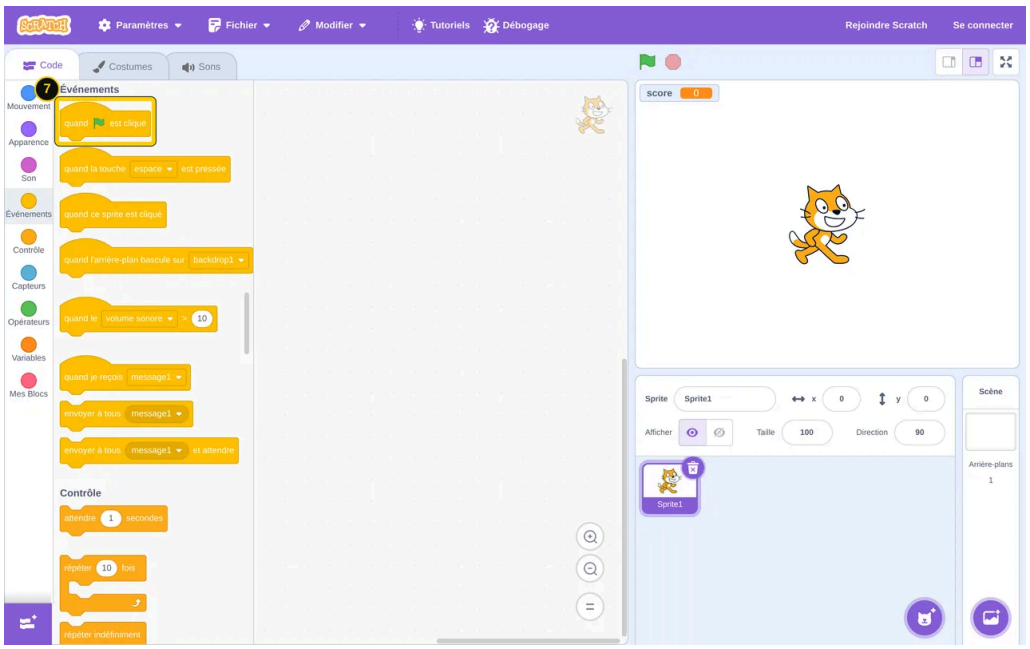
Étape 6. Cliquez sur la catégorie Événements (rond jaune).

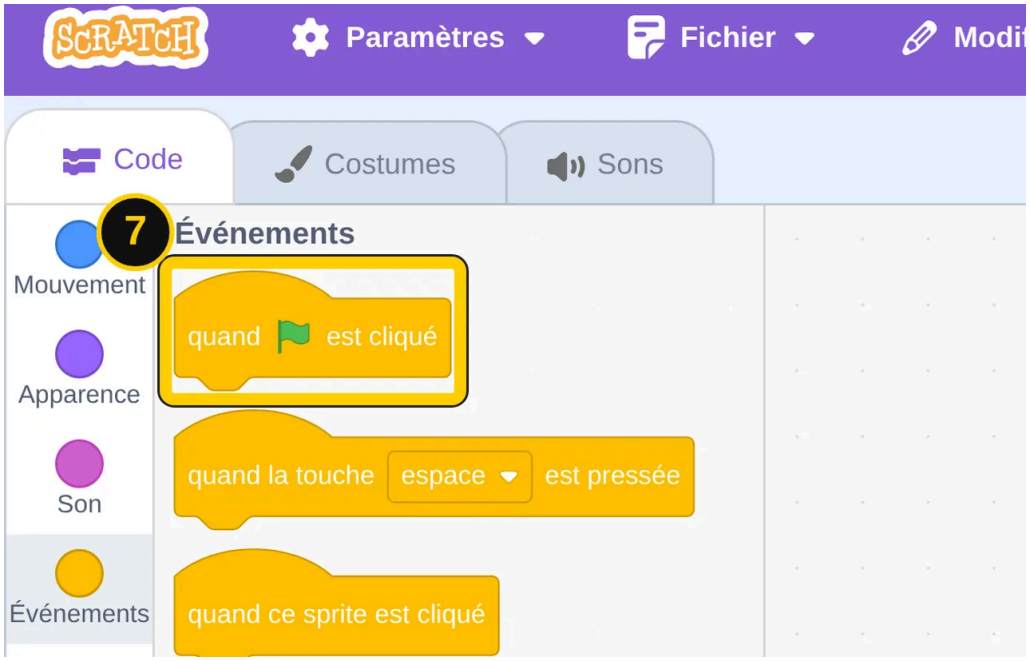




Vous devez voir : les blocs jaunes des événements ; le premier est « quand ⚑ est cliqué ».

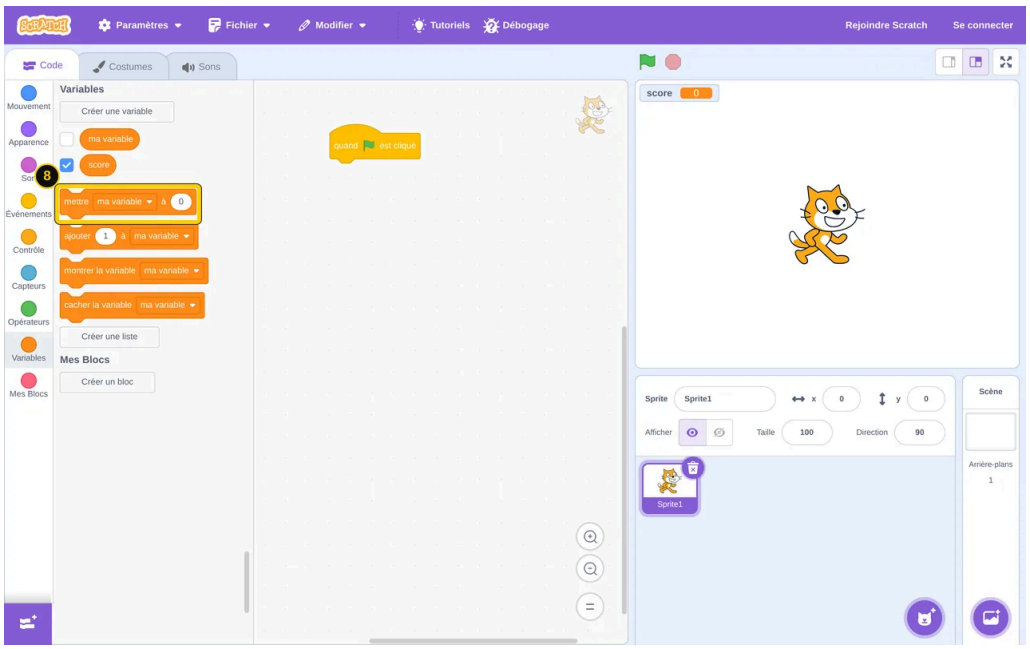
Étape 7. Glissez le bloc « quand ⚑ est cliqué » dans la zone de travail, au centre.





Vous devez voir : le bloc « quand ⚑ est cliqué » dans la zone de travail.

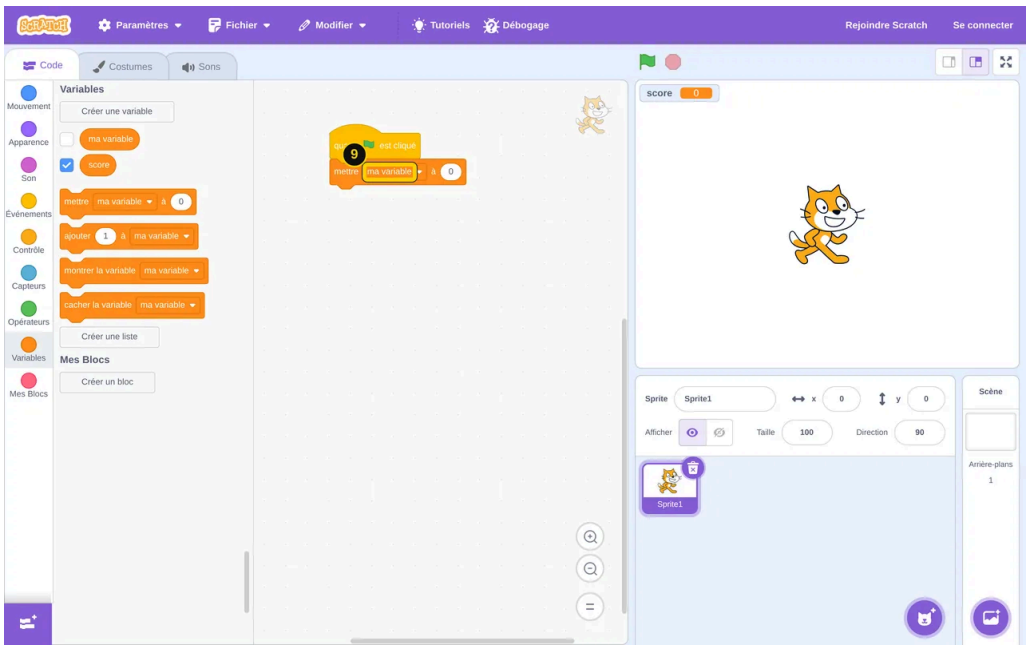
Étape 8. Revenez dans la catégorie **Variables**, et glissez le bloc « mettre ma variable à 0 » juste sous « quand ⚑ est cliqué ».





Vous devez voir : les deux blocs emboîtés l'un sous l'autre. S'ils restent séparés, rapprochez le second jusqu'à ce qu'il s'emboîte.

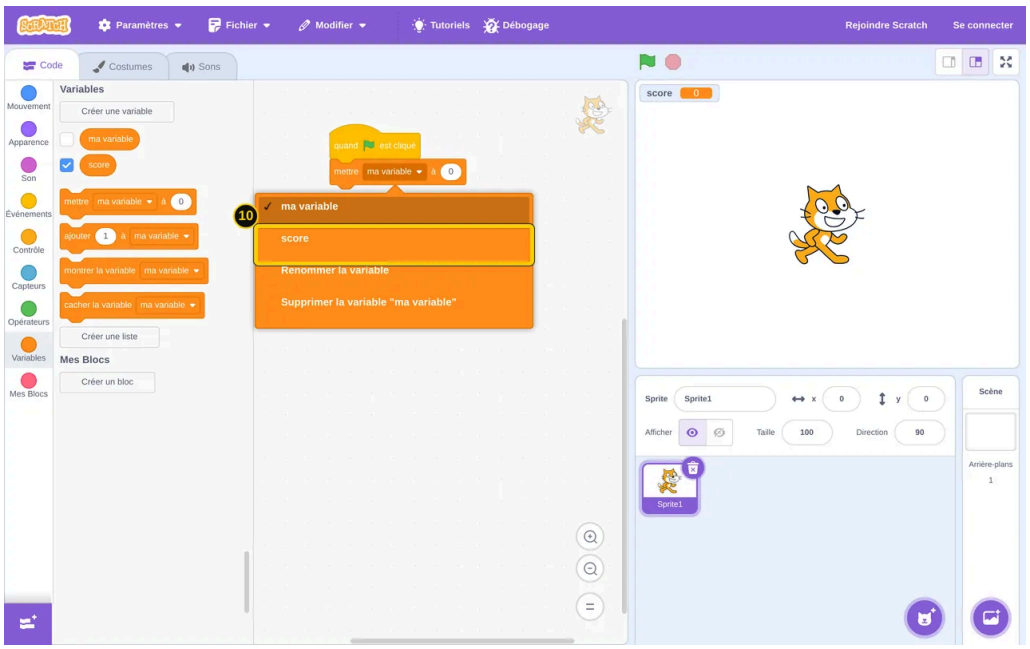
Étape 9. Dans le bloc posé, cliquez sur la petite flèche à droite de « ma variable ».

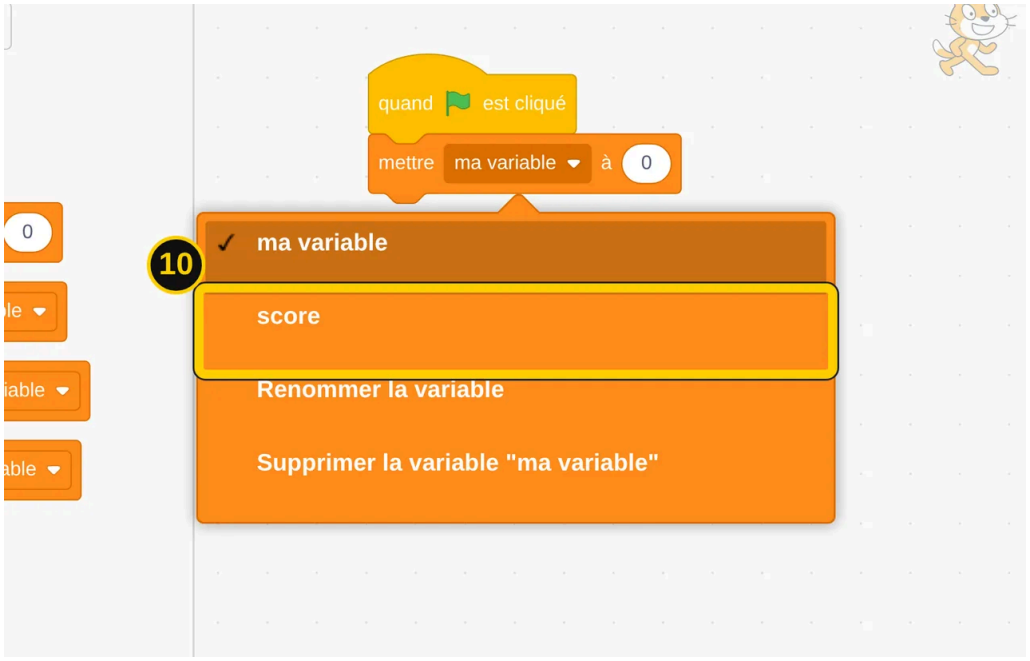




Vous devez voir : une liste : « ma variable », « score », « Renommer la variable », « Supprimer la variable ».

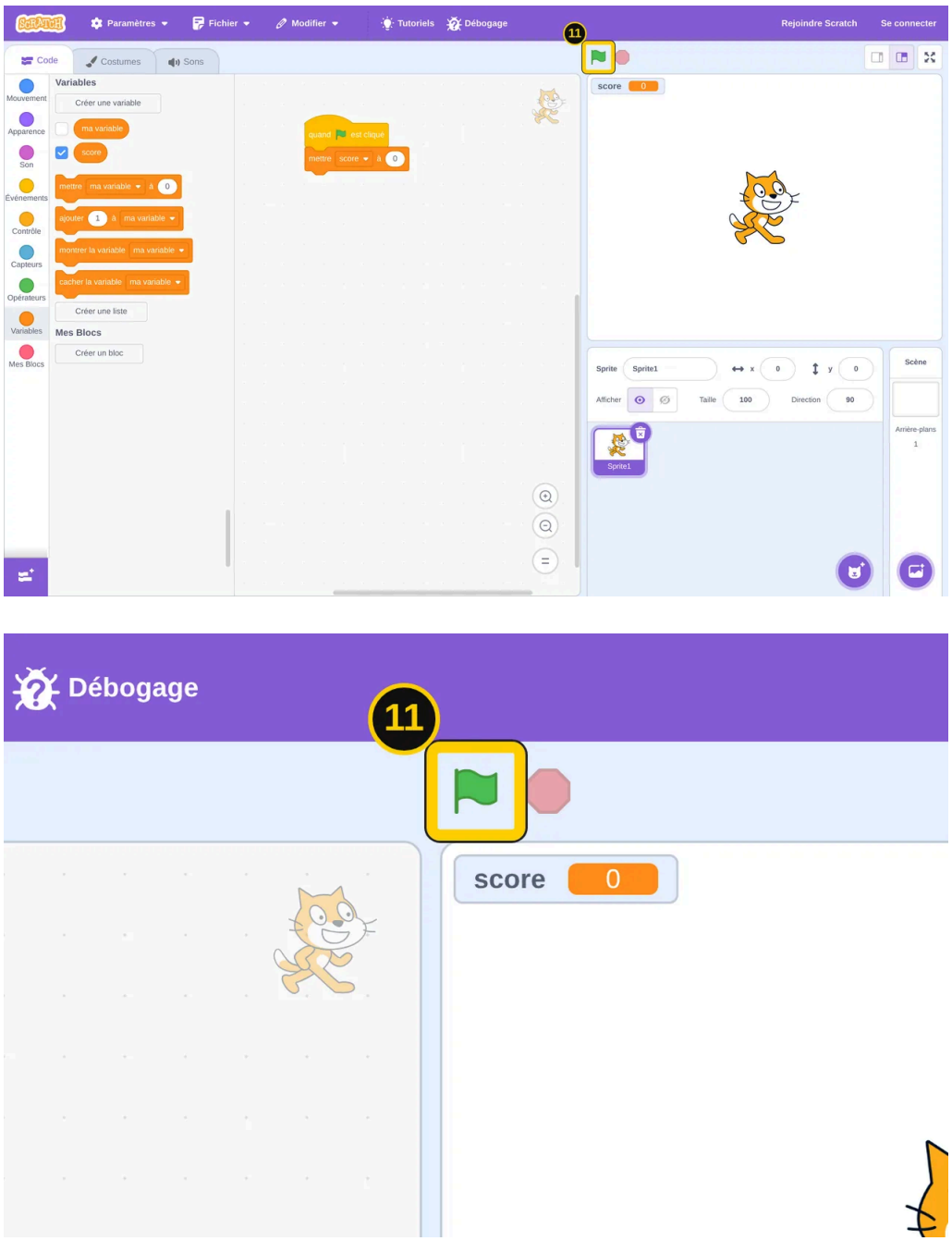
Étape 10. Dans la liste, cliquez sur **score**.





Vous devez voir : le bloc dit maintenant « mettre score à 0 ».

Étape 11. Cliquez sur le drapeau vert, au-dessus de la scène.



Vous devez voir : le script s'entoure d'un contour jaune pendant qu'il s'exécute, et la scène affiche `score 0`.

Captures de l'éditeur Scratch, cadres compris, sous licence CC BY-SA 2.0 : « Scratch is developed by the Lifelong Kindergarten Group at the MIT Media Lab. See http://scratch.mit.edu ».

Variables Scratch — attention et vérification

> ⚠ Attention
>
> Une variable créée « pour ce sprite uniquement » est invisible des autres sprites. Si deux sprites l'utilisent, créez-la « pour tous les sprites ».

- **Vérifiez** : `score` à 5, puis « ajouter –2 » : combien ?

Exercices 3 à 5 — Scratch (45 min)

1. Ensemble (5 min) : l'exercice 3, le score.
2. Seuls : l'exercice 4, le compte à rebours.
3. Seuls : l'exercice 5, devine le nombre.
4. **Réussi si** : chaque programme fait ce qu'annonce son énoncé.

Exercice 3 — Le score (facile). Au drapeau vert : « mettre score à 0 ». Chaque clic sur le sprite ajoute 1 au score. **Vérifiez** : le score s'affiche et repart à 0 au drapeau vert.

Exercice 4 — Le compte à rebours (moyen).

1. Une variable `compte` part de 10.
2. Le sprite dit la valeur pendant 1 seconde, puis on retire 1.
3. On recommence jusqu'à ce que `compte` vaille 0.
4. Il dit alors « Décollage ! ».

Exercice 5 — Devine le nombre (moyen+).

1. Le programme choisit un nombre secret entre 1 et 100.
2. Il demande « Votre nombre ? » et répond « plus grand » ou « plus petit ».
3. Il recommence jusqu'à ce que le joueur trouve.
4. **Bonus** : il compte les essais et les annonce à la fin.

> 💡 Indice — exercice 3
>
> L'événement « quand ce sprite est cliqué » se trouve dans la catégorie Événements. Cochez la variable `score` pour la voir sur la scène.

> **Indice — exercice 4**
>
> Utilisez « répéter jusqu'à ce que » avec la condition « compte = 0 » (catégorie Opérateurs), et « ajouter –1 à compte » pour diminuer. C'est le compte à rebours de la semaine 1… sans le bug.

> **Indice — exercice 5**
>
> « mettre secret à nombre aléatoire entre 1 et 100 », puis une boucle « répéter jusqu'à ce que réponse = secret ». Dans la boucle : « demander … et attendre », puis un bloc « si … alors … sinon » qui compare « réponse » et « secret ».

Si le temps le permet — « Deux pages à la fois »

1. Écrivez le pseudo-code de « deux pages à la fois ».
2. Prévoyez le cas où l'on a **dépassé** le nom.
3. Comptez les pages au pire, pour 1 000 et pour 2 000 pages.

À retenir — ce matin

> **À retenir**
>
> - **Pseudo-code** : une instruction par ligne ; prévoir le cas « absent ».
> - **Couper en deux** : on élimine la moitié à chaque essai.
> - **Variable** : un nom qui désigne une valeur ; « pour tous les sprites ».

Le détail, notion par notion :

- **Pseudo-code** : du français structuré, avant le vrai code ; une instruction par ligne ; ce qui est « à l'intérieur » est décalé ; un bon algorithme prévoit l'échec.
- **Couper en deux** : la même méthode trouve un nom dans un annuaire et un nombre entre 1 et 100 ; 7 essais suffisent pour 100 nombres.

- **Variable** : un nom qui désigne une valeur ; « mettre » donne une valeur, « ajouter » la modifie ; créez-la « pour tous les sprites » si plusieurs sprites l'utilisent.

Débrief

- Annuaire et devine le nombre : quel point commun ?
- Pourquoi un algorithme doit-il prévoir le cas « absent » ?
- À quoi sert une variable ? Donnez un exemple hors informatique.

Après-midi — TP — « Problem set 0 : projet Scratch »

Apport 3 — Tout est 0 et 1 (15 min)

- **Pourquoi** : comprendre ce que l'ordinateur manipule vraiment.
- **Binaire** : compter avec seulement deux chiffres, 0 et 1.

Pourquoi. Un texte, une image, un programme : pour l'ordinateur, tout est fait de nombres, et ces nombres sont écrits avec des 0 et des 1.

Définition. Le binaire est une façon d'écrire les nombres avec deux chiffres seulement, 0 et 1 (des **bits**) : on compte en base 2 au lieu de la base 10 (CS50 2026a). Les cartes de CS Unplugged le font toucher du doigt : chaque carte vaut le double de la suivante, et chaque nombre s'écrit en retournant certaines cartes (CS Unplugged, s. d.).

Exemple. Cinq cartes de 16, 8, 4, 2 et 1 points. Carte visible = 1, carte cachée = 0. Pour 9 = 8 + 1 :

```
cartes :  16   8   4   2   1
9      :   0   1   0   0   1
```

Pas à pas. Par deux, avec les cinq cartes (fournies par le formateur, ou dessinées sur cinq bouts de papier). L'un montre les cartes, l'autre vérifie le total, puis échangez.

1. Posez les cartes face cachée, de gauche à droite : 16, 8, 4, 2, 1.
2. Pour 9 : retournez 8, puis 1. Le total vaut 9 : on écrit `01001`.
3. Montrez 21, puis 31. Écrivez chaque nombre avec des 0 et des 1.

4. Que vaut `01001` ?
5. Quel est le plus grand nombre avec 5 cartes ? Et avec une carte de plus ?

Tout le reste est construit sur des nombres. Un **texte** : chaque caractère a un numéro (la lettre `A` porte le numéro 65 dans le code ASCII, repris par Unicode, qui numérote les caractères de toutes les langues et les émojis). Une **image** : une grille de points, les pixels ; chaque pixel est fait de trois nombres, la quantité de rouge, de vert et de bleu (RGB) (CS50 2026a). Vous retrouverez ces trois nombres en CSS, en semaine 5. Pour aller plus loin, facultatif : la fiche de référence Systèmes de numération (décimal, binaire, hexadécimal).

Binaire — attention et vérification

> ⚠ Attention
>
> On lit les cartes de gauche à droite, de la plus grande à la plus petite. Changer leur ordre change le nombre.

- **Vérifiez** : que vaut `00110` ?

Apport 4 — Dans l'ordinateur (5 min)

- **Pourquoi** : savoir où vit votre programme quand il tourne.
- **Processeur, mémoire, disque** : exécuter, poser, garder.

Pourquoi. Quand vous lancez un programme, il passe du disque à la mémoire, puis le processeur l'exécute. Savoir qui fait quoi explique, par exemple, pourquoi un travail non enregistré disparaît quand l'ordinateur s'éteint.

Définition. Le **processeur** exécute les instructions d'un programme, une par une, dans l'ordre ; le programme et ses données sont placés dans la **mémoire vive**, qui s'efface quand l'ordinateur n'est plus alimenté (Delacroix-Gouin 2019) ; le **disque** garde les fichiers quand l'ordinateur est éteint : c'est là qu'est enregistré votre projet Scratch.

Une image, avec sa limite. On peut se représenter l'ordinateur comme une cuisine (schéma ci-dessous). Limite de l'image : un

vrai processeur suit des milliards d'instructions par seconde, et plusieurs programmes partagent la mémoire en même temps.

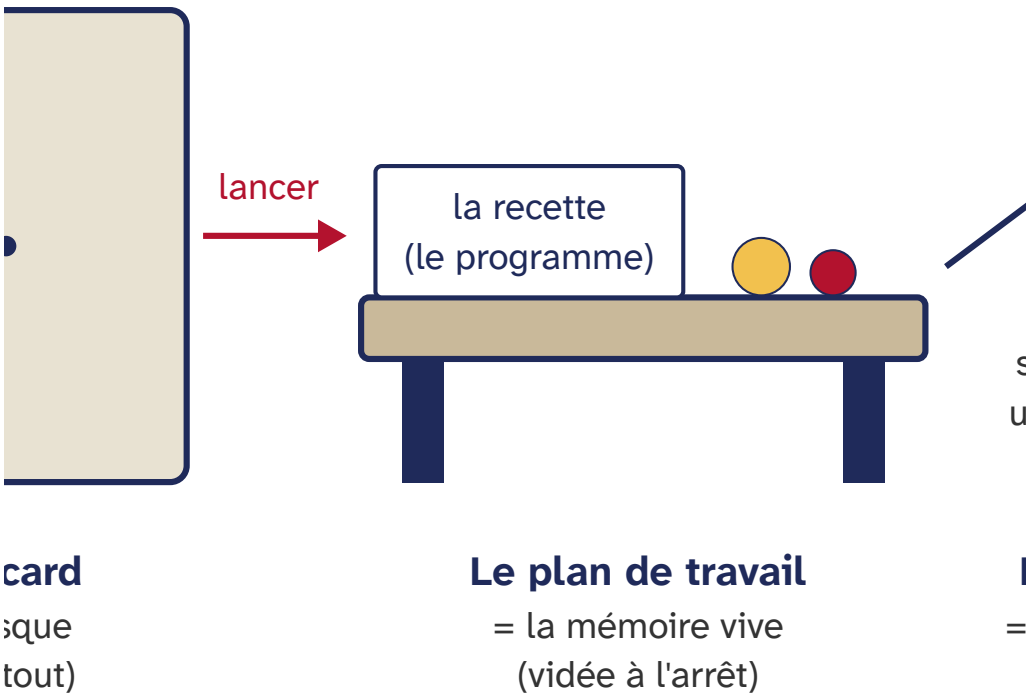


Figure 1. – Une image : l'ordinateur comme une cuisine. Lancer un programme, c'est sortir la recette du placard pour la poser sur le plan de travail, où le cuisinier la suit.

Pas à pas. Suivez le trajet de votre projet Scratch :

1. Vous ouvrez le fichier `.sb3` : il est lu sur le **disque**.
2. Il est copié dans la **mémoire vive**.
3. Le **processeur** exécute ses blocs, un à la fois.
4. Vous enregistrez : la nouvelle version est écrite sur le **disque**.

C'est tout ce qu'il faut savoir du matériel pour la formation. Pour aller plus loin, facultatif : la fiche de lecture C'est quoi un ordinateur ? (environ 20 minutes).

Dans l'ordinateur — attention et vérification

> ⚠ Attention
>
> Ce qui n'est pas enregistré n'existe qu'en mémoire vive : une coupure de courant l'efface. Enregistrez votre projet souvent.

- **Vérifiez** : où se trouve votre projet quand l'ordinateur est éteint ?

Apport 5 — Les messages entre sprites (10 min)

- **Pourquoi** : qu'un sprite déclenche l'action d'un autre.
- **Message** : un signal envoyé à tous, reçu par ceux qui l'attendent.

Pourquoi. Dans un jeu, quand le chat touche la balle, le score doit changer et un autre sprite doit réagir. Chaque sprite a ses propres scripts : il faut un moyen de les faire communiquer. Le cahier des charges de votre projet exige un message.

Définition. Un message est un signal nommé : « envoyer à tous » l'émet ; chaque script qui commence par « quand je reçois » ce même nom démarre à ce moment-là (catégorie Événements) (Scratch Foundation, s. d.-b).

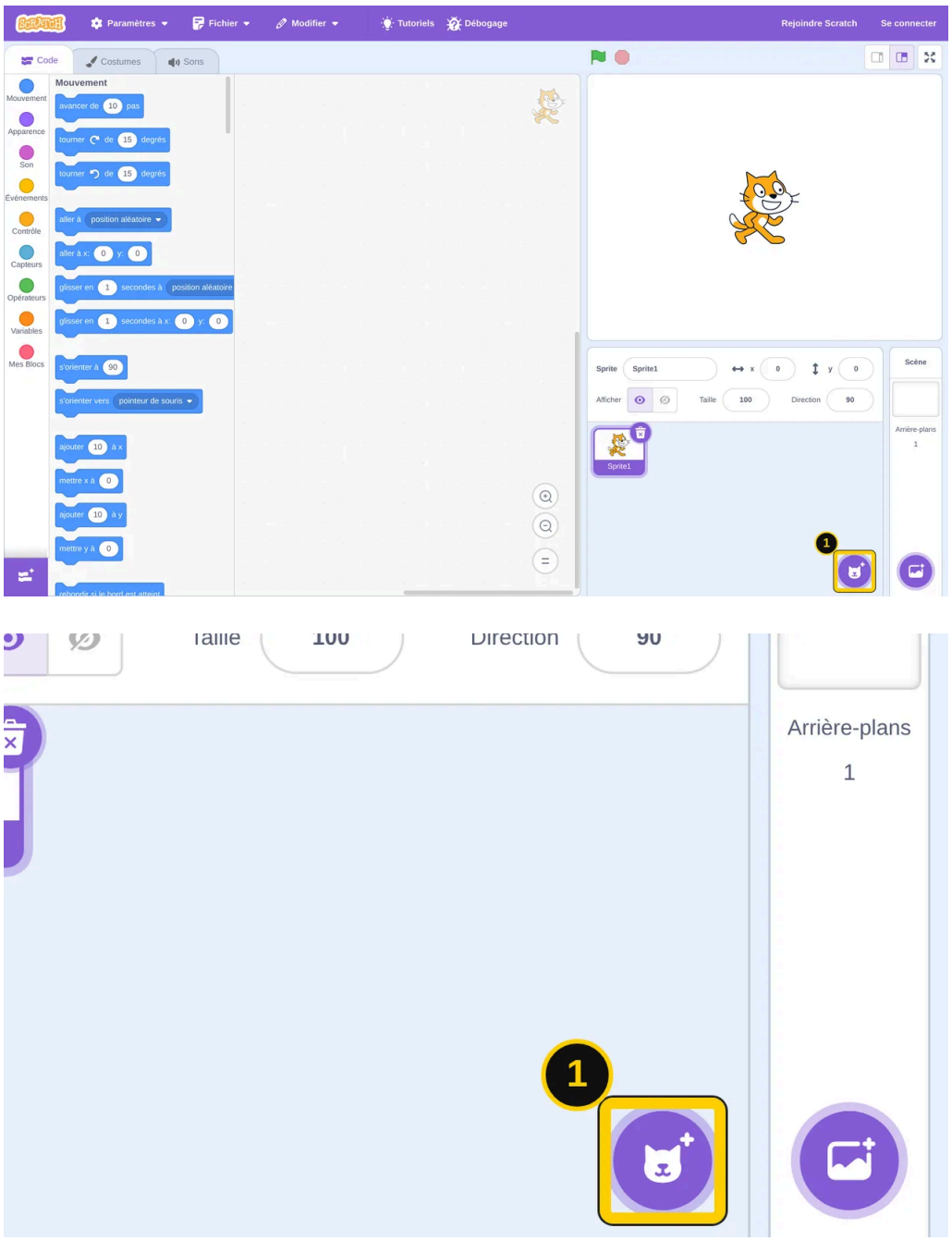
Exemple. Deux sprites, deux scripts :

```
Sprite « Chat » :
quand ⚑ est cliqué
attendre 1 secondes
envoyer à tous [départ]

Sprite « Balle » :
quand je reçois [départ]
dire [C'est parti !] pendant 2 secondes
```

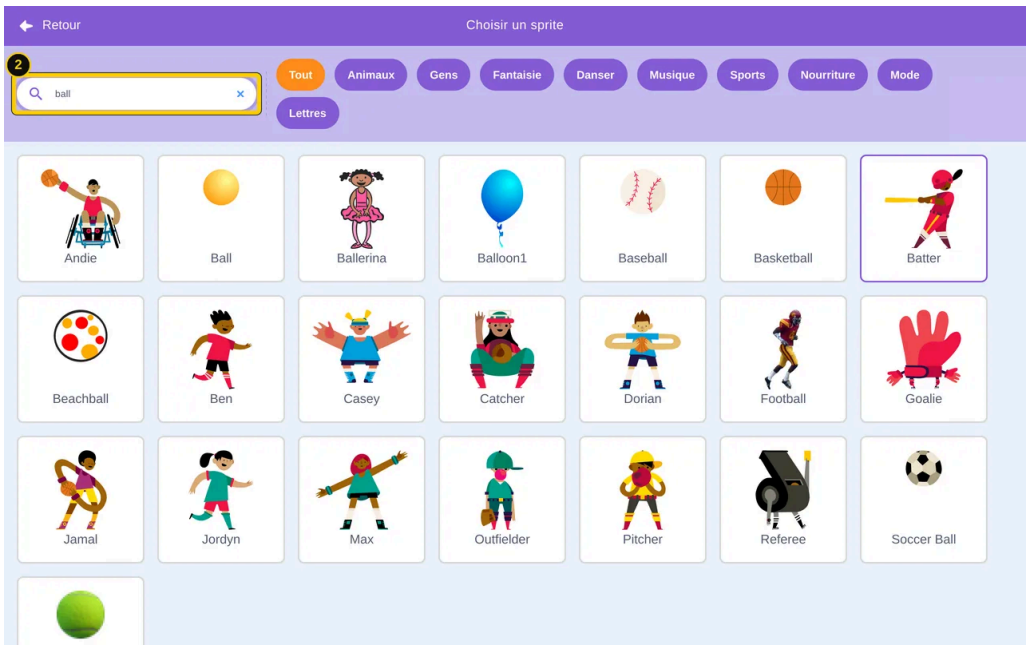
Pas à pas. Le formateur fait chaque étape ; vous la refaites aussitôt dans Scratch. Sur chaque capture, l'endroit à cliquer est entouré d'un cadre jaune, avec le numéro de l'étape ; un zoom sur la zone utile suit chaque capture. Sur téléphone, touchez une capture pour l'agrandir.

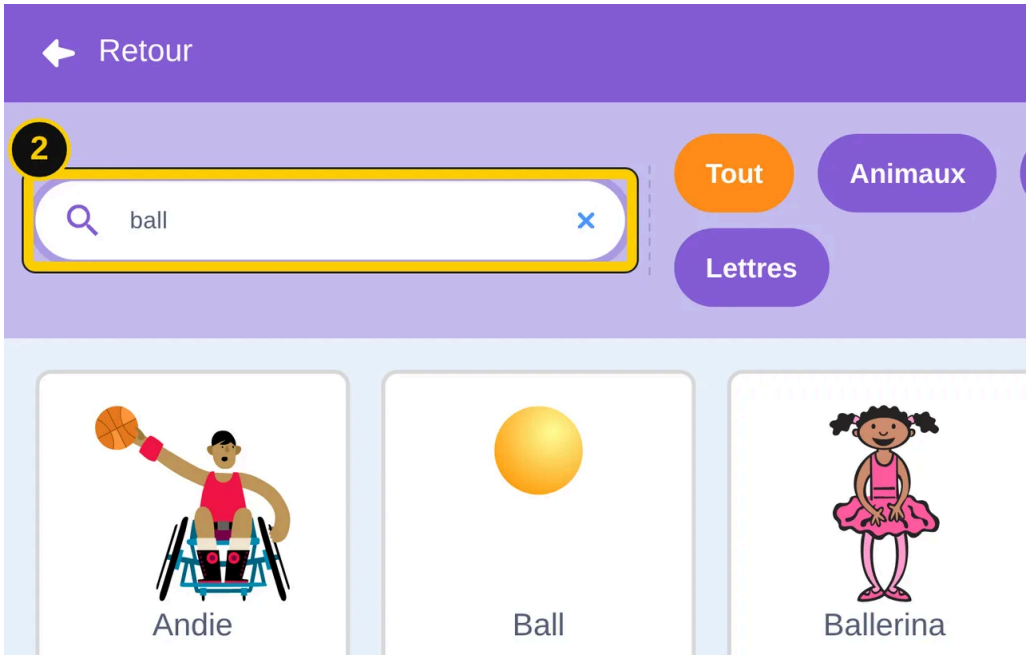
Étape 1. En bas à droite, sous la liste des sprites, cliquez sur le bouton rond **Choisir un sprite** (une tête de chat avec un « + »).



Vous devez voir : la bibliothèque « Choisir un sprite ».

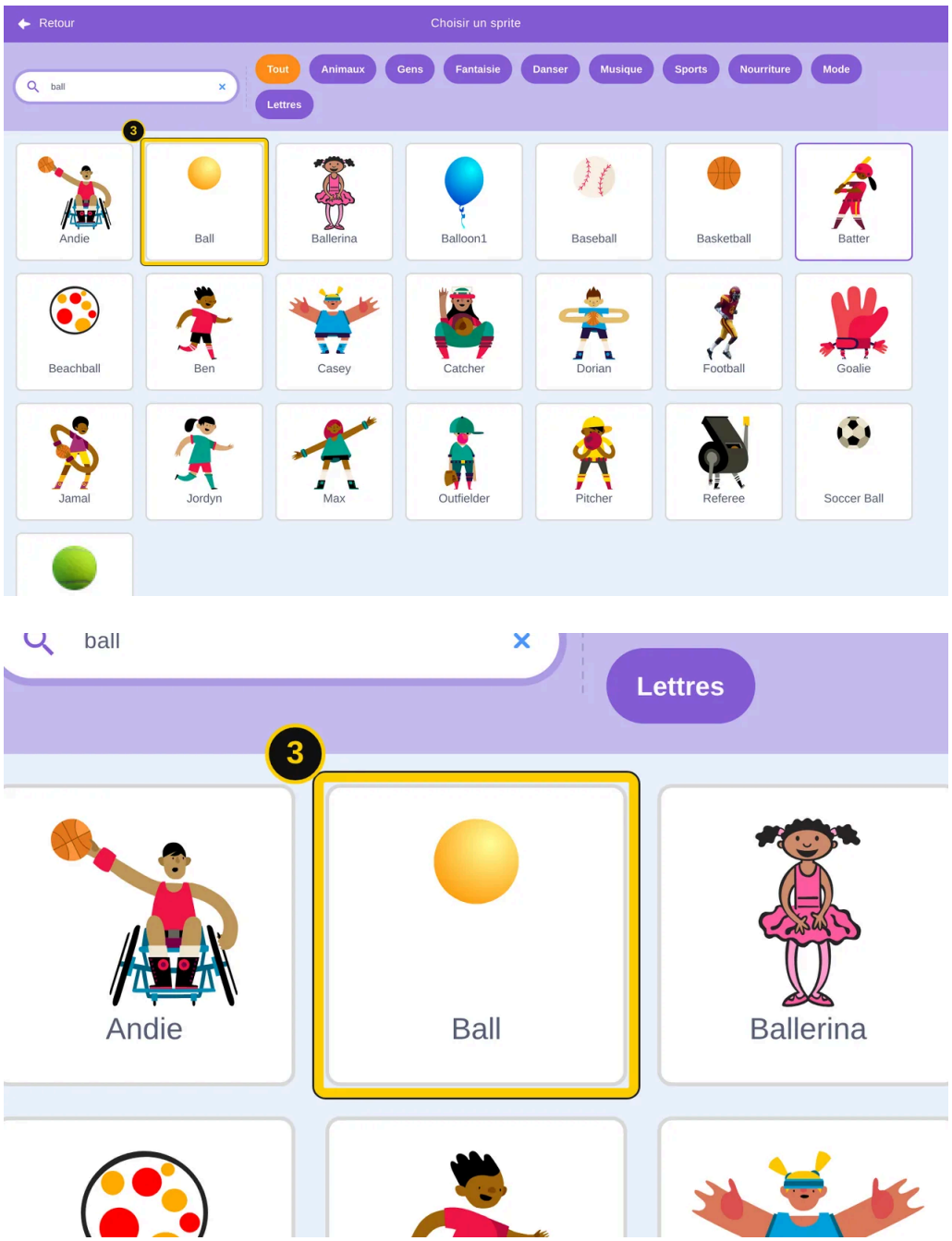
Étape 2. Dans le champ de recherche, en haut à gauche, tapez `ball`.





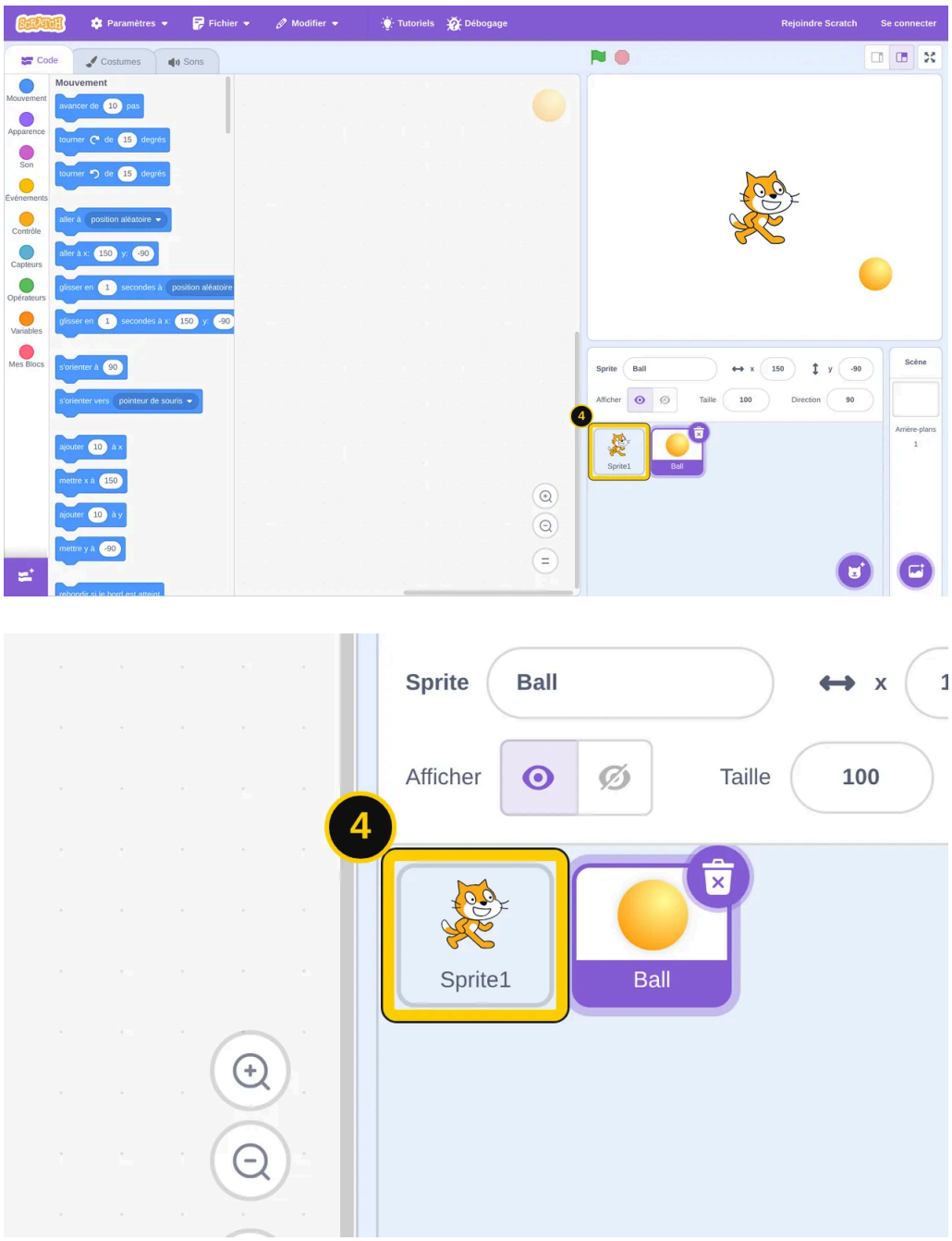
Vous devez voir : les sprites dont le nom contient « ball » : Ball, Ballerina, Baseball, Basketball...

Étape 3. Cliquez sur **Ball**, la balle jaune.



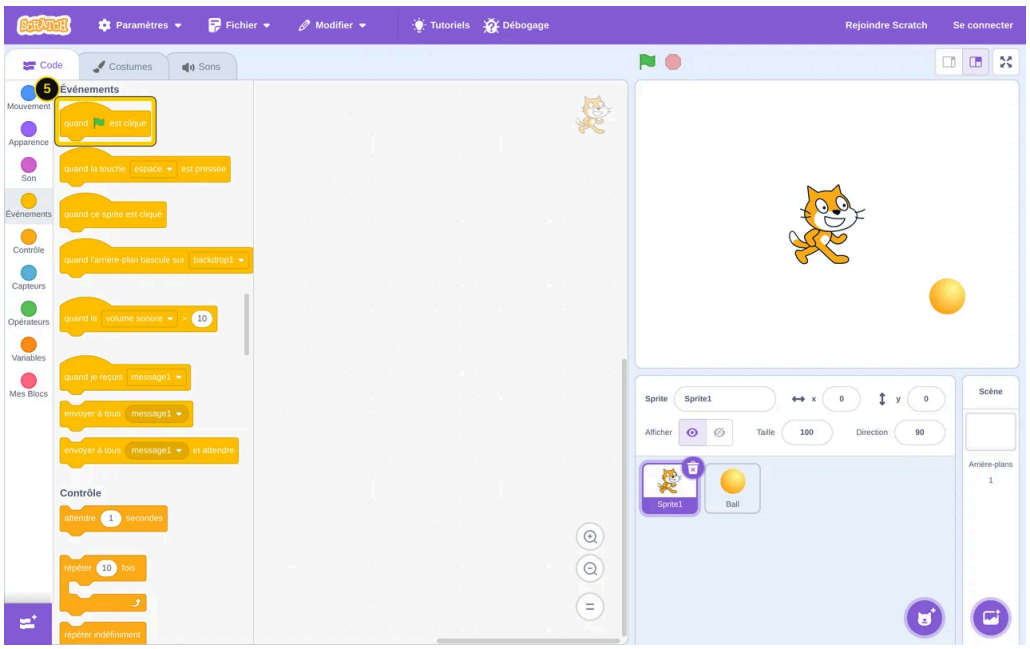
Vous devez voir : la balle sur la scène, et « Ball » dans la liste des sprites. Scratch la pose à un endroit au hasard : sa place peut différer de la capture.

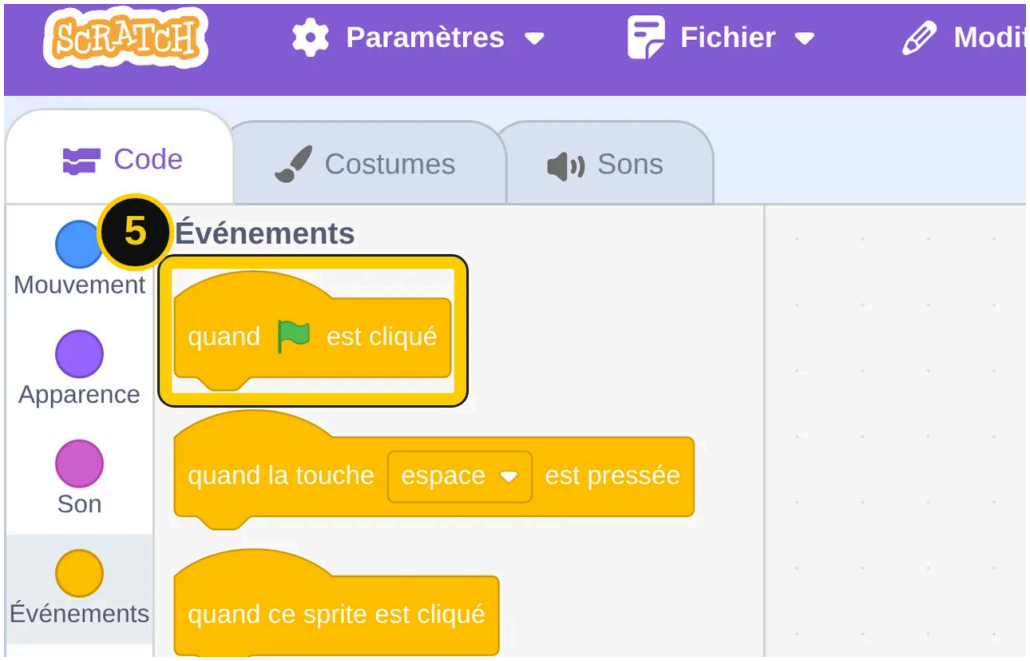
Étape 4. Dans la liste des sprites, cliquez sur **Sprite1** (le chat).



Vous devez voir : Sprite1 entouré en bleu ; la zone de travail montre les scripts du chat.

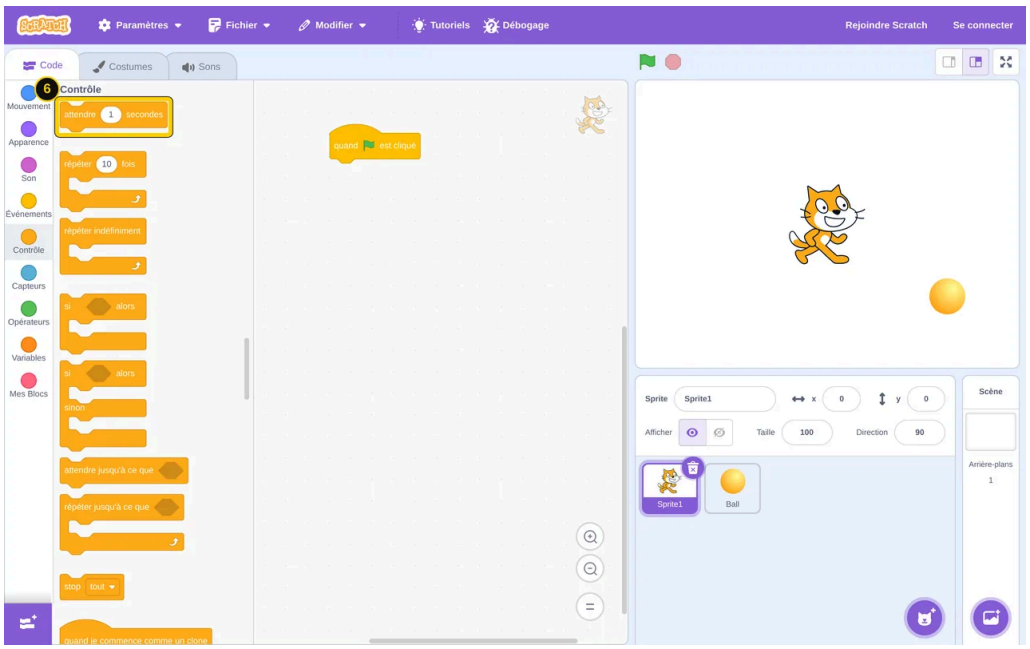
Étape 5. Dans la catégorie Événements, glissez « quand ⚑ est cliqué » dans la zone de travail.





Vous devez voir : le bloc dans la zone de travail.

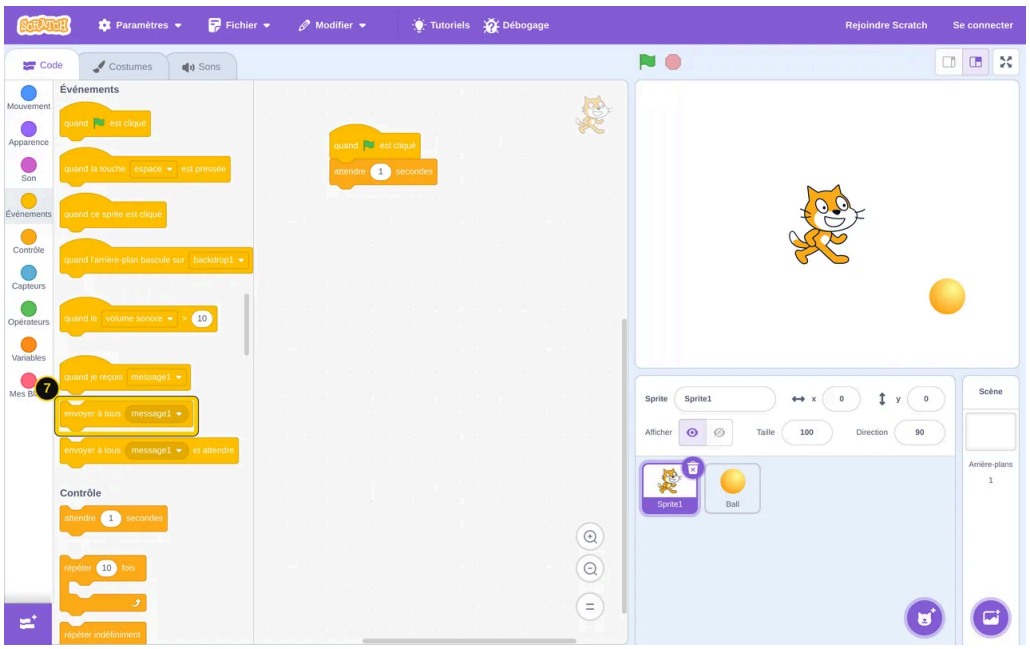
Étape 6. Dans la catégorie **Contrôle** (rond orange clair), glissez « attendre 1 secondes » sous « quand ⚑ est cliqué ».

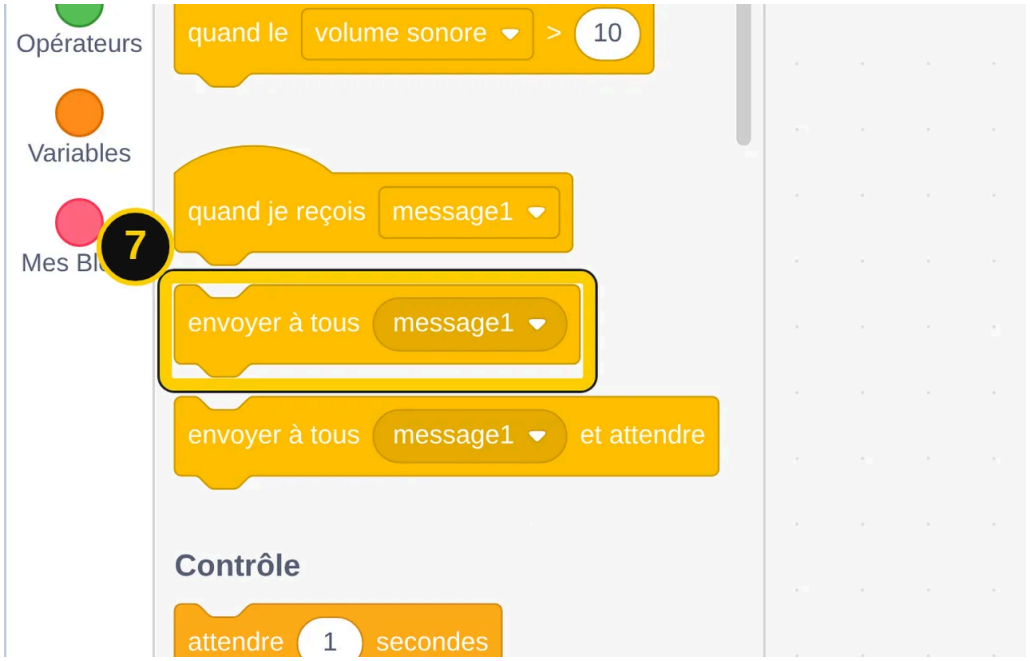




Vous devez voir : les deux blocs emboîtés.

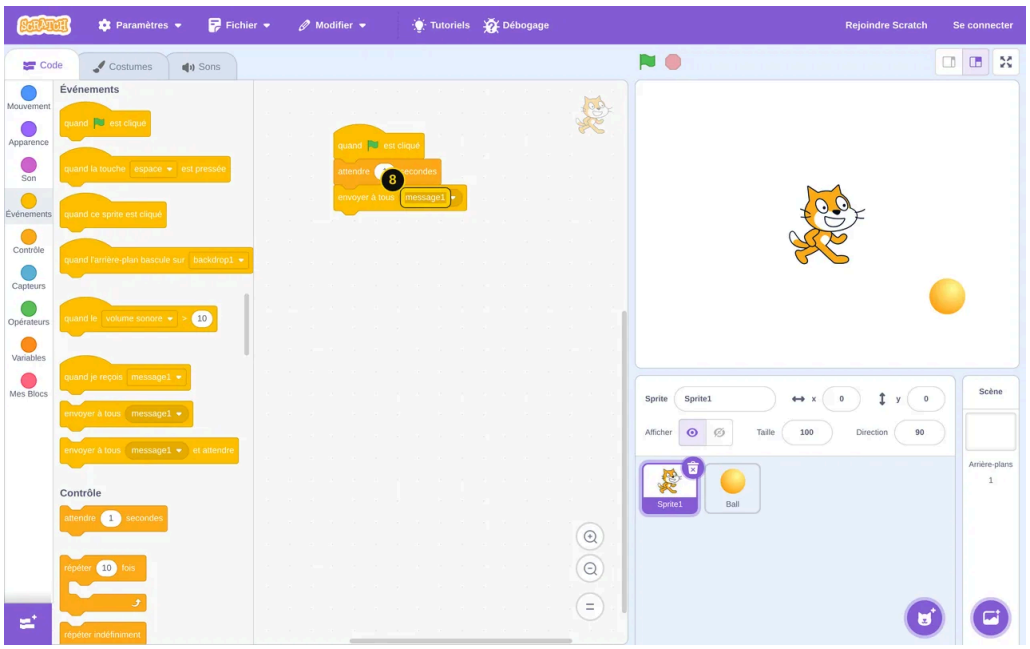
Étape 7. Dans la catégorie Événements, glissez « envoyer à tous message1 » sous « attendre 1 secondes ».

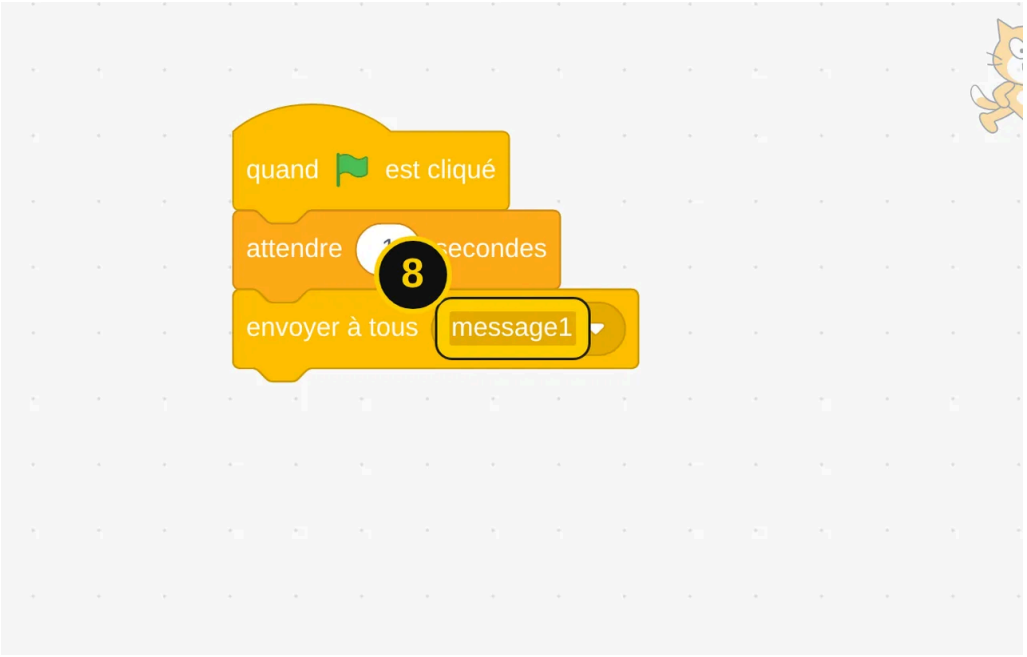




Vous devez voir : trois blocs emboîtés.

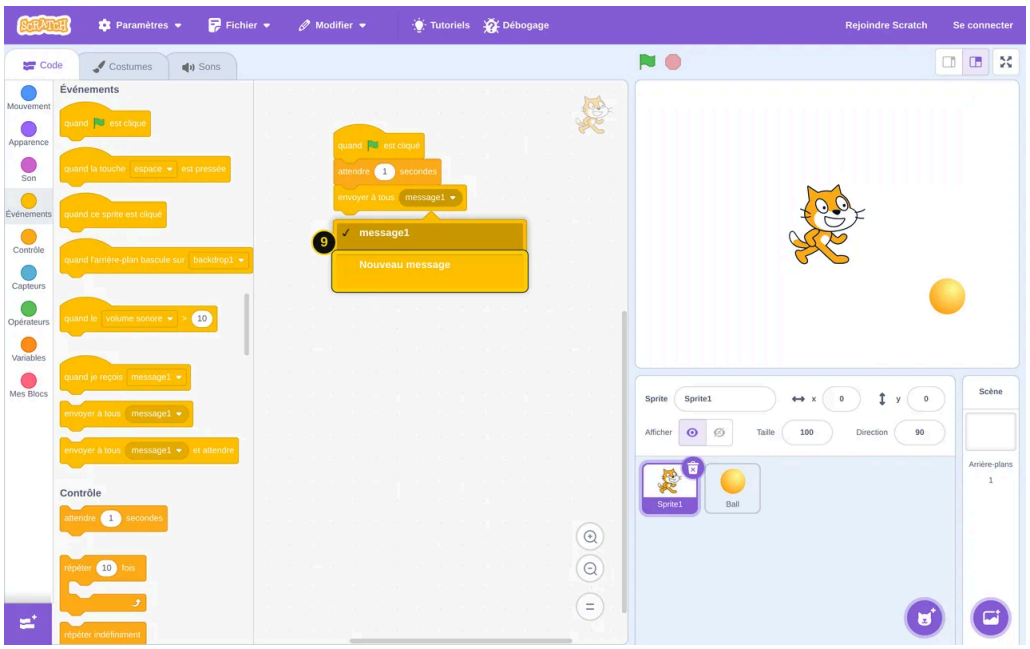
Étape 8. Dans ce bloc, cliquez sur la petite flèche à droite de « message1 ».

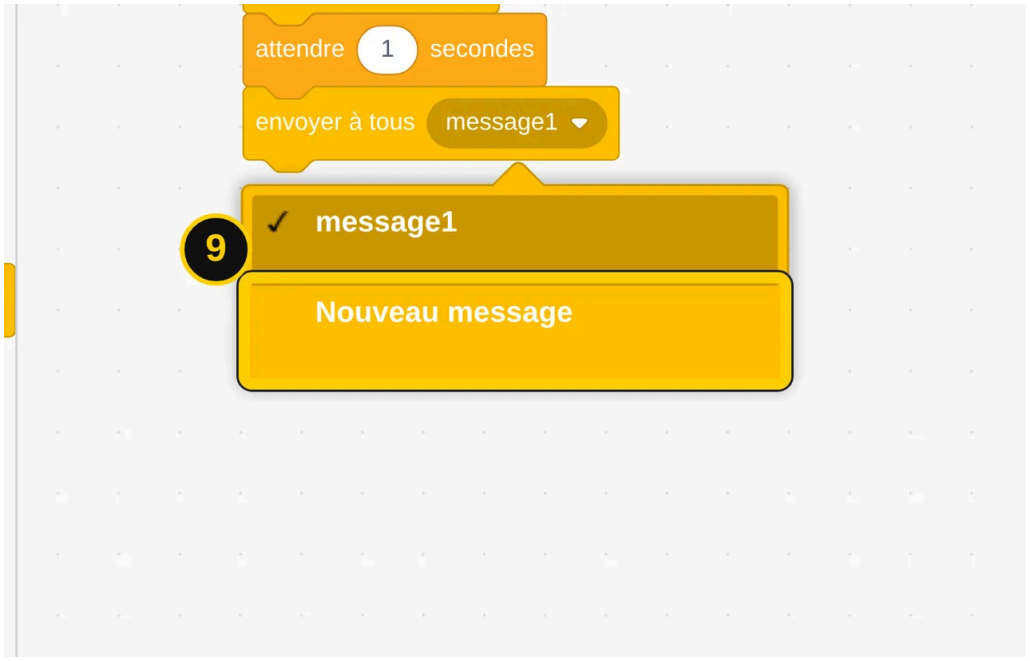




Vous devez voir : une liste : « message1 » et **Nouveau message**.

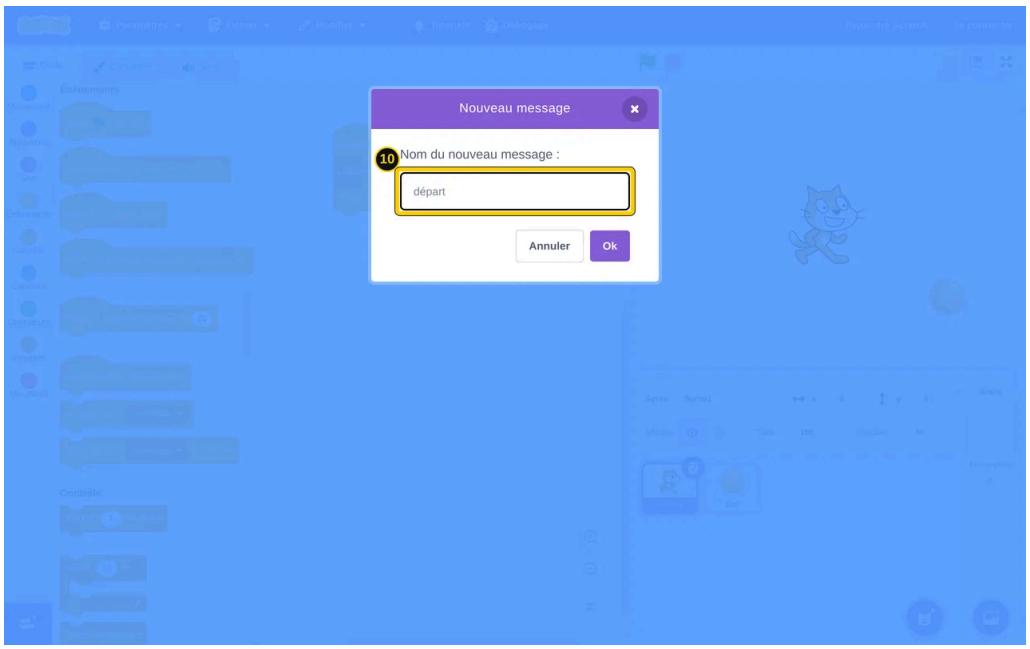
Étape 9. Cliquez sur **Nouveau message**.

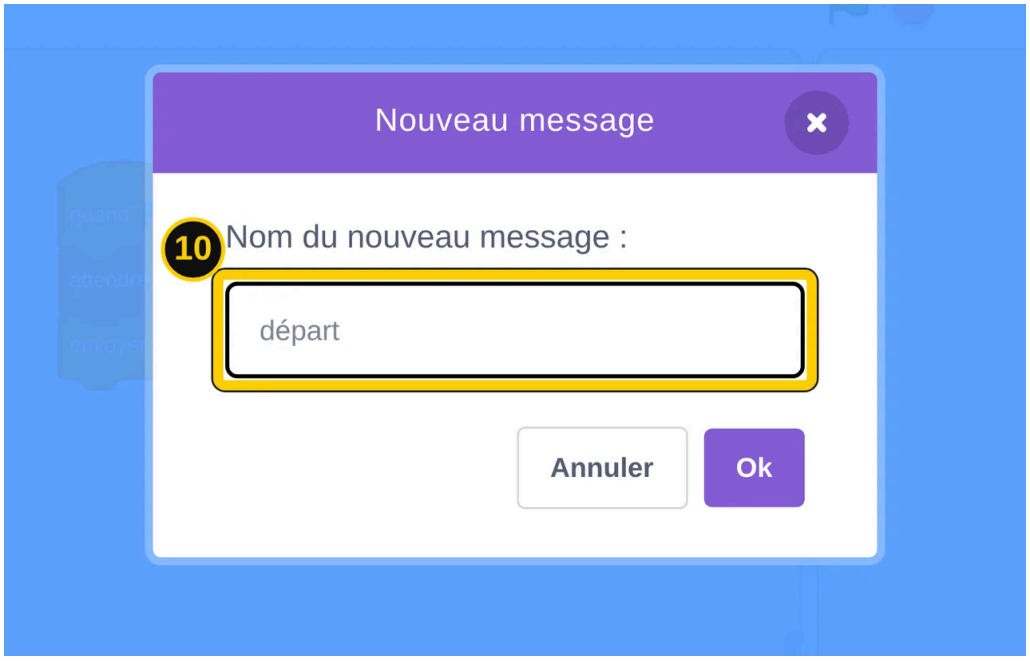




Vous devez voir : la fenêtre « Nouveau message ».

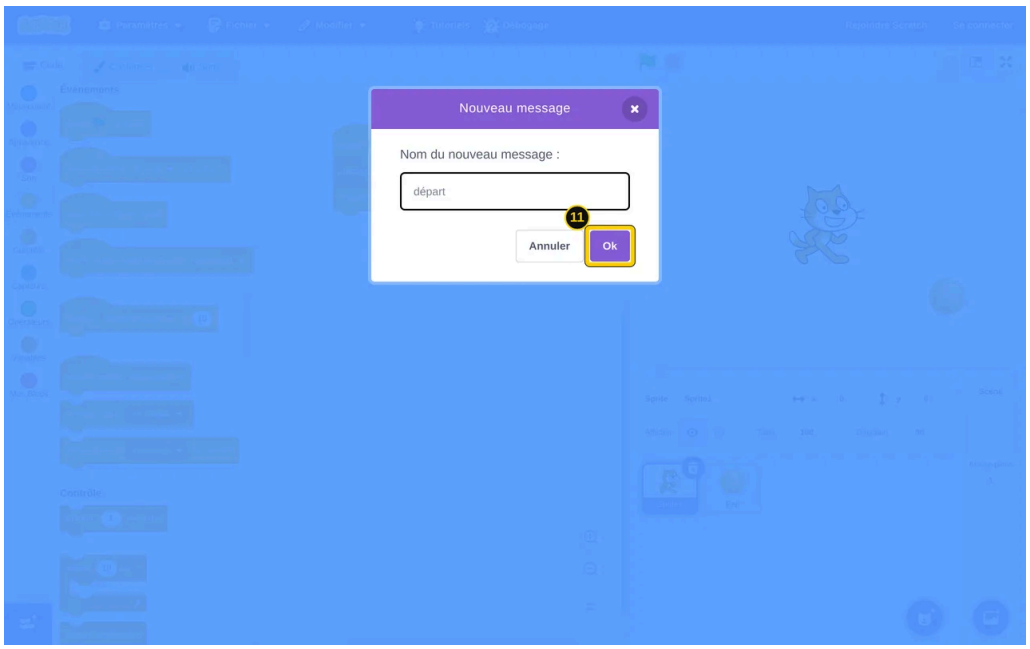
Étape 10. Dans le champ « Nom du nouveau message », tapez `départ`.

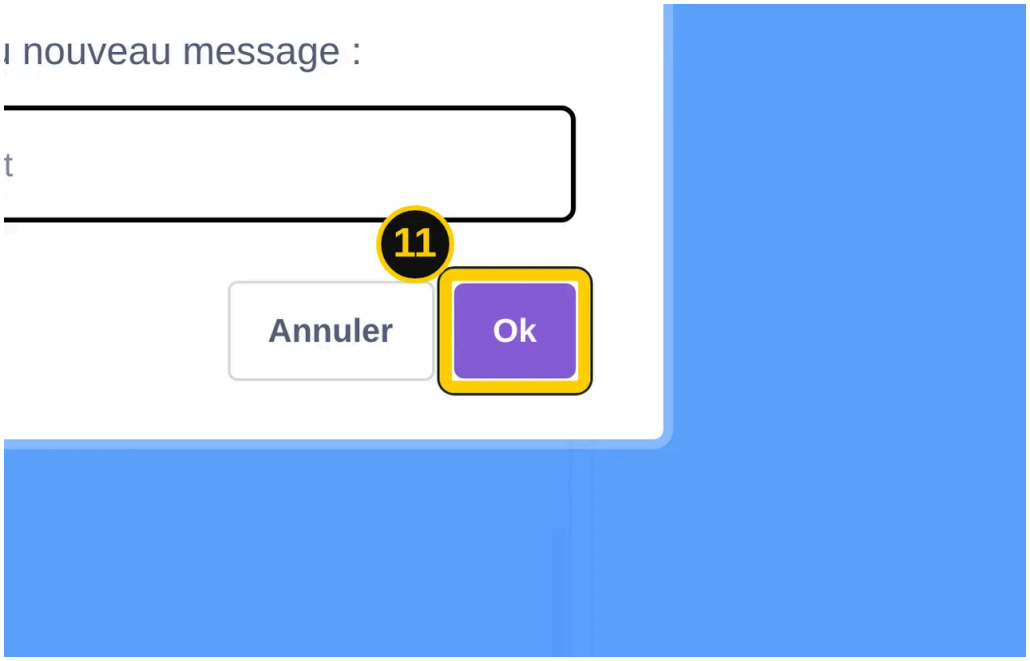




Vous devez voir : `départ` dans le champ.

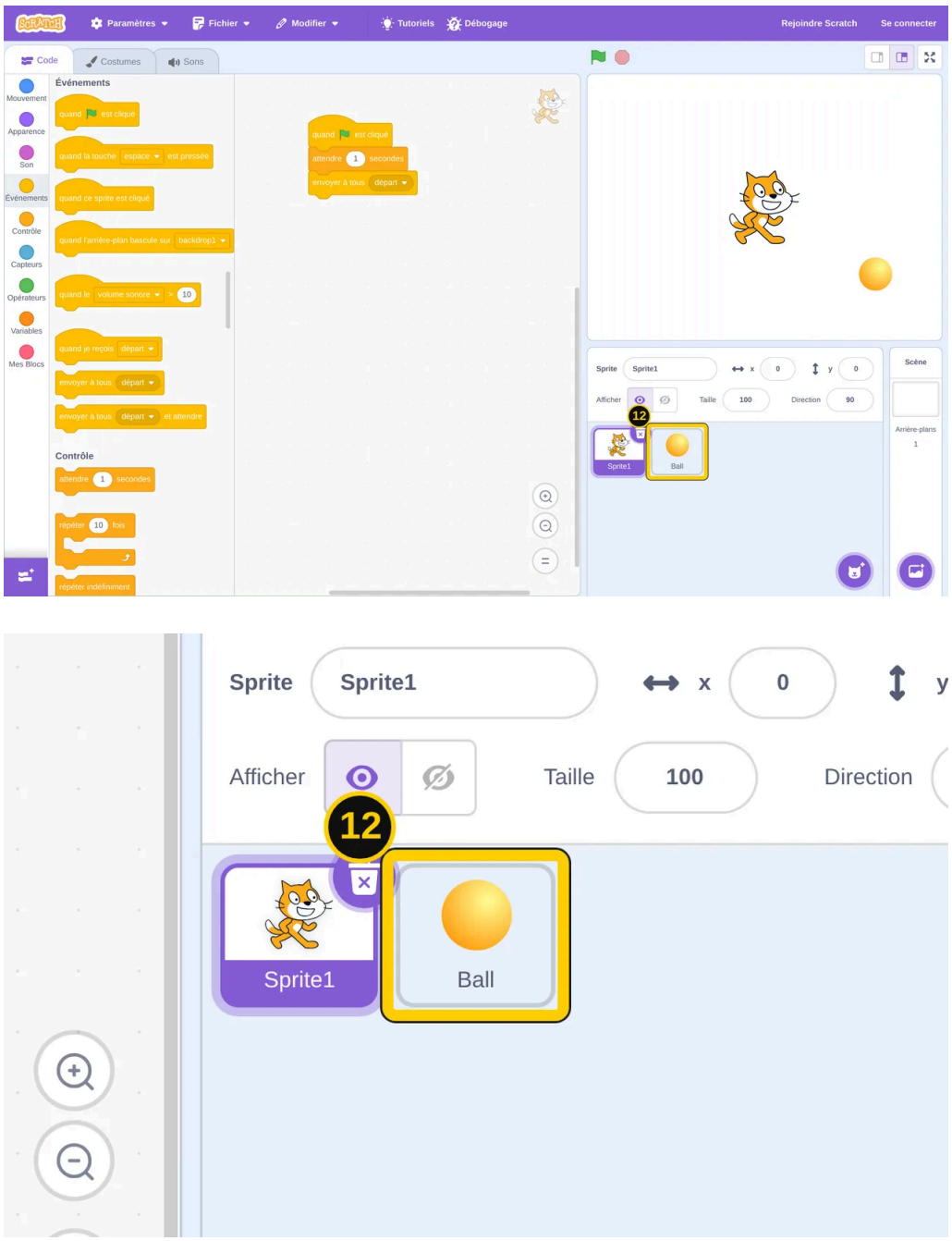
Étape 11. Cliquez sur **Ok**.





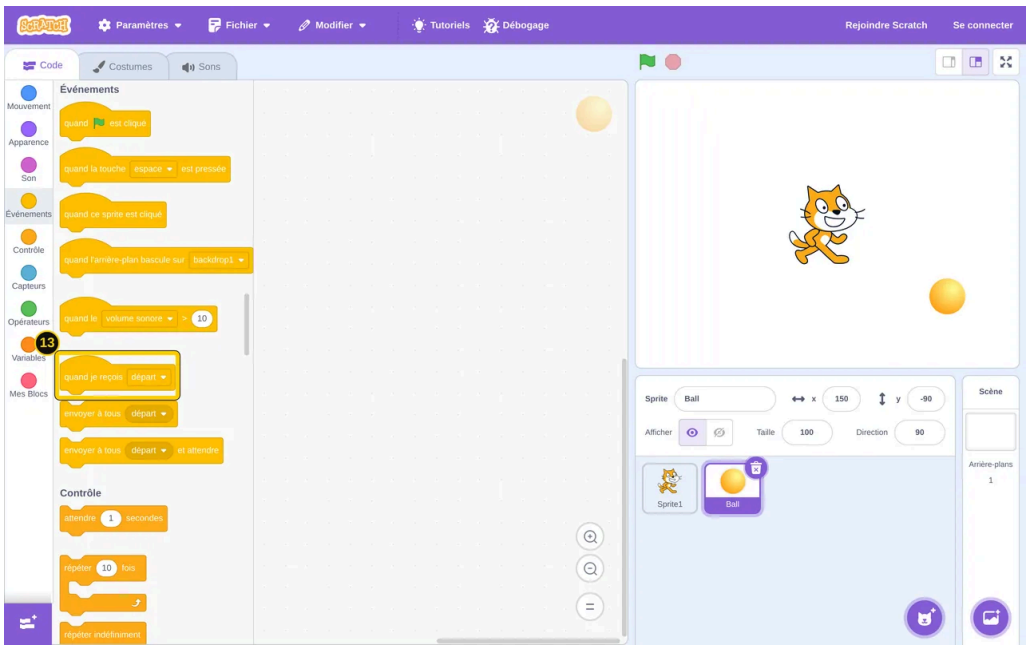
Vous devez voir : le dernier bloc dit « envoyer à tous départ ».

Étape 12. Dans la liste des sprites, cliquez sur **Ball**.



Vous devez voir : Ball entouré en bleu ; la zone de travail de la balle, vide.

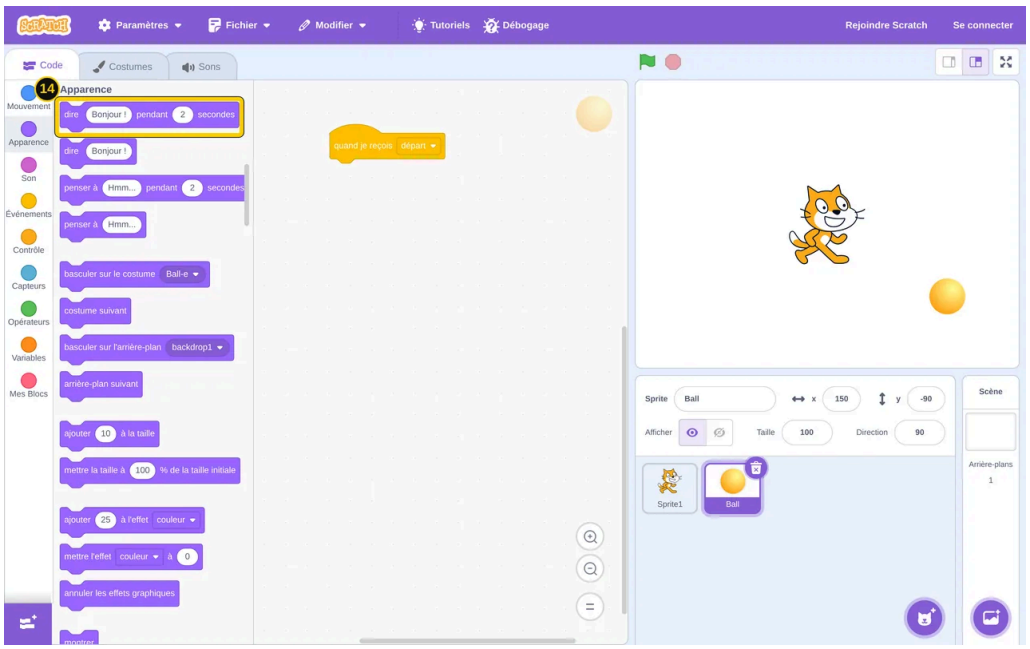
Étape 13. Dans la catégorie Événements, glissez « quand je reçois départ » dans la zone de travail. Si le bloc affiche un autre nom, choisissez `départ` dans sa liste.

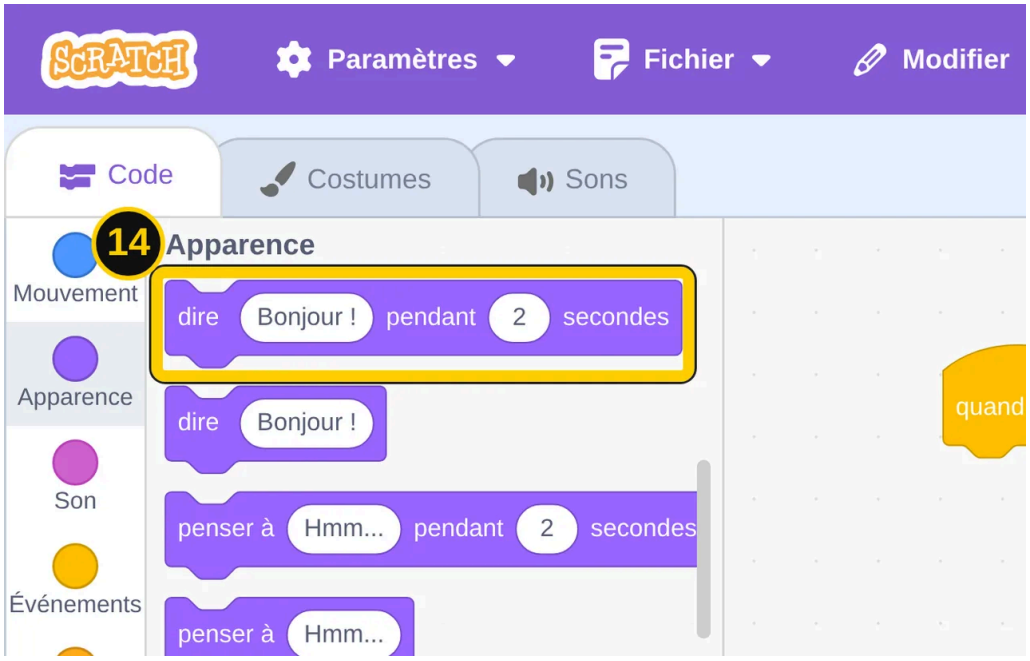




Vous devez voir : le bloc « quand je reçois départ » dans la zone de travail de la balle.

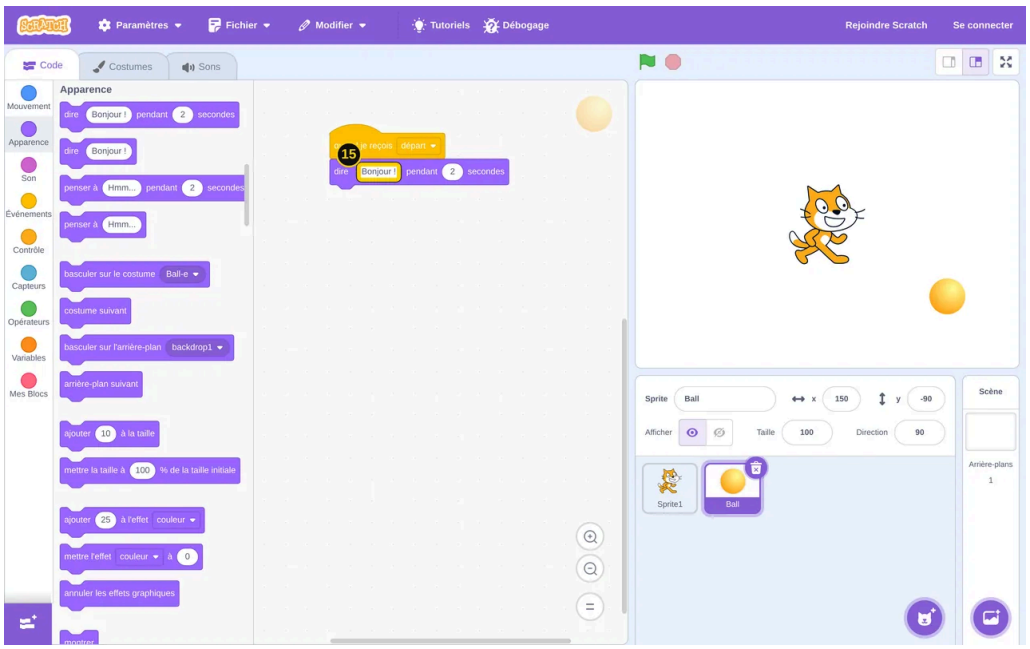
Étape 14. Dans la catégorie **Apparence** (rond violet), glissez « dire Bonjour ! pendant 2 secondes » sous « quand je reçois départ ».

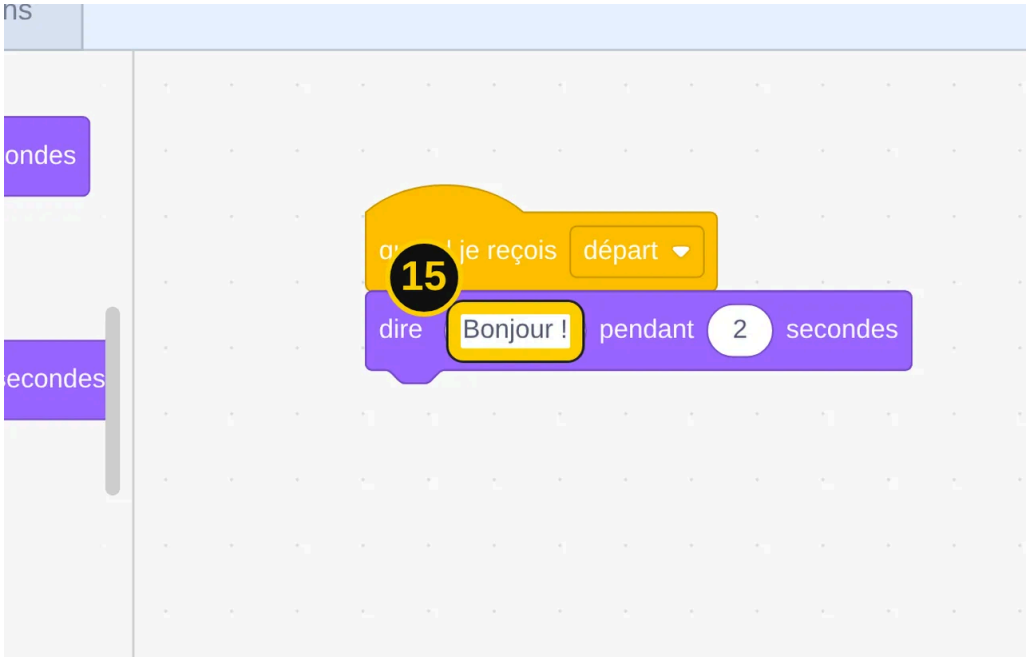




Vous devez voir : les deux blocs emboîtés.

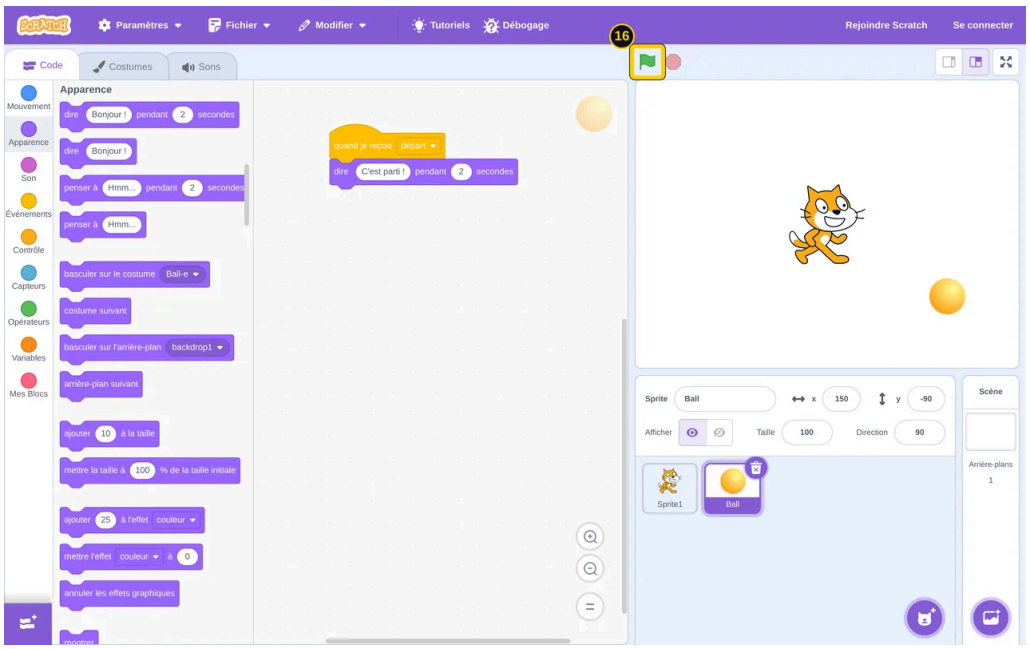
Étape 15. Dans le bloc posé, cliquez sur le texte « Bonjour ! », remplacez-le par `C'est parti !`, puis appuyez sur Entrée.

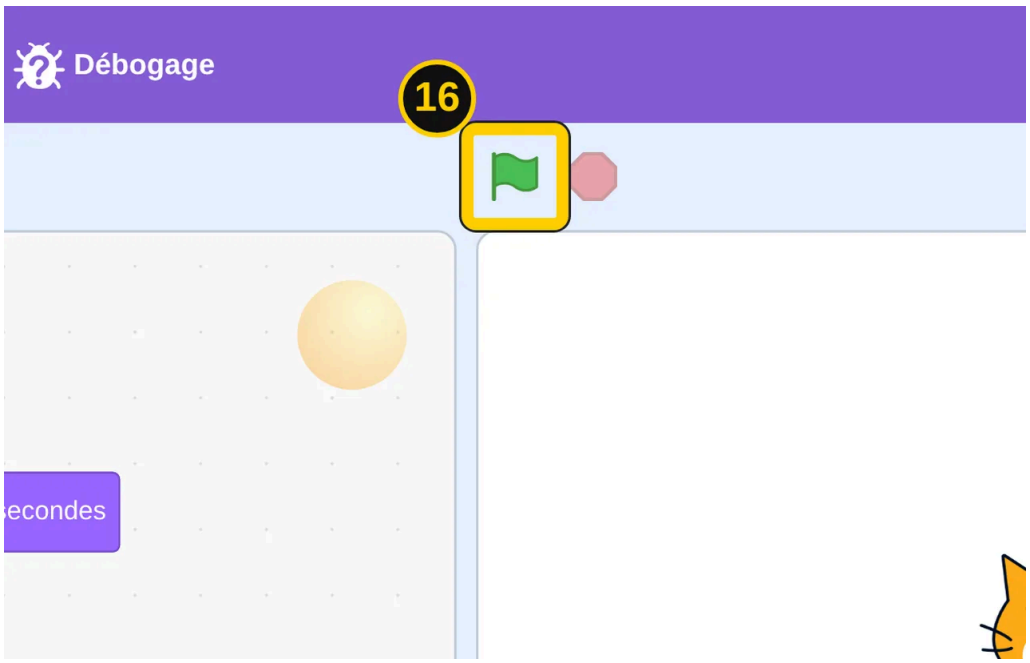




Vous devez voir : le bloc dit « dire C'est parti ! pendant 2 secondes ».

Étape 16. Cliquez sur le drapeau vert.





Vous devez voir : une seconde après le clic, la balle affiche la bulle « C'est parti ! » pendant 2 secondes.

Captures de l'éditeur Scratch, cadres compris, sous licence CC BY-SA 2.0 : « Scratch is developed by the Lifelong Kindergarten Group at the MIT Media Lab. See http://scratch.mit.edu ».

Messages — attention et vérification

> ⚠ Attention
>
> Le message ne fait rien ? Le nom doit être exactement le même dans « envoyer à tous » et « quand je reçois ». Choisissez-le dans la liste, ne le retapez pas.

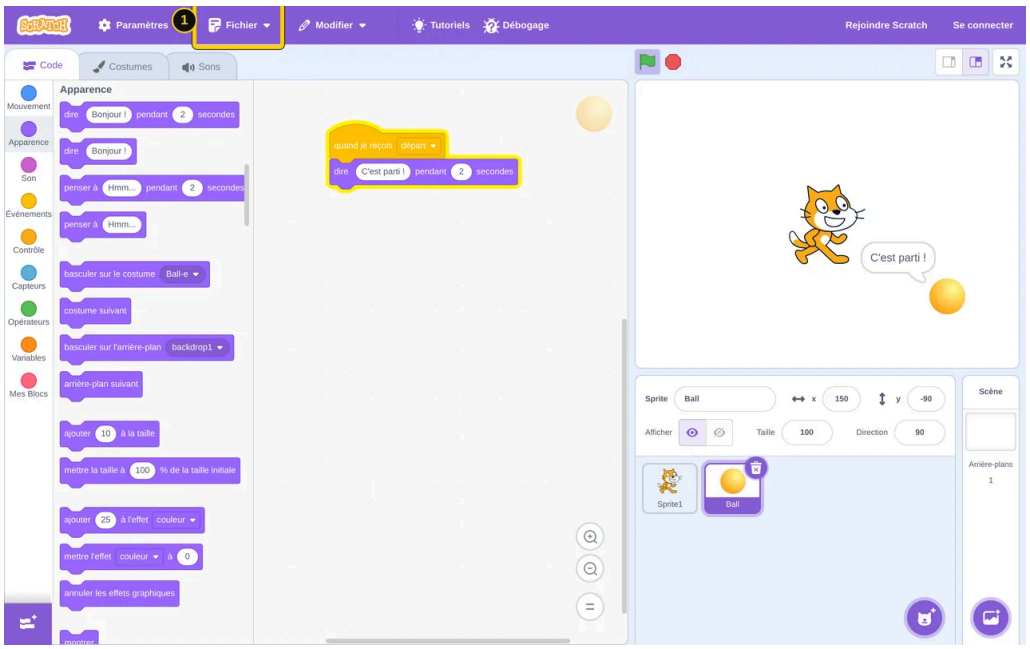
- **Vérifiez** : ajoutez un troisième sprite qui réagit aussi à `départ`.

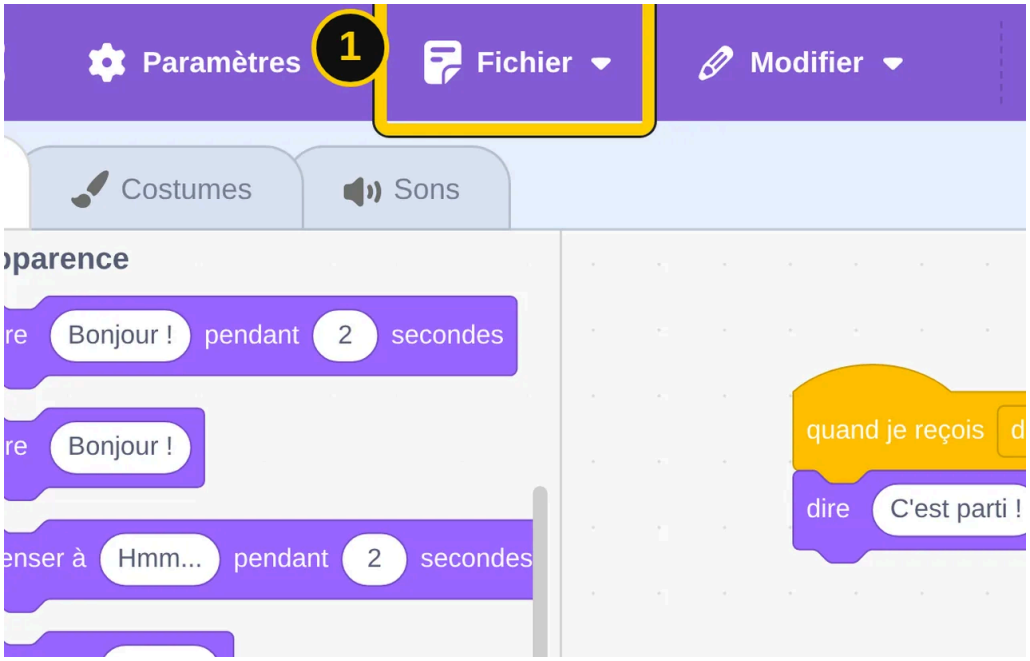
Le projet — consignes (2 h 10)

1. Écrivez votre idée en trois phrases, sur papier (10 min).
2. Créez un **jeu** ou une **histoire interactive**, sur le thème de votre choix.
3. Par étapes : ajouter, tester, sauvegarder.
4. Cochez au fur et à mesure le cahier des charges, ci-dessous.
5. Un compte Scratch n'est **pas obligatoire**, ni la publication.

Ce TP s'inspire du premier problème de CS50, *Starting from Scratch* (CS50 2026b), avec un cahier des charges propre à la formation. Sans compte, travaillez dans l'éditeur en ligne et enregistrez régulièrement le fichier : menu **Fichier**, puis **Sauvegarder sur votre ordinateur** (Scratch Foundation, s. d.-a). Le projet se termine mercredi matin : ce soir, il est commencé.

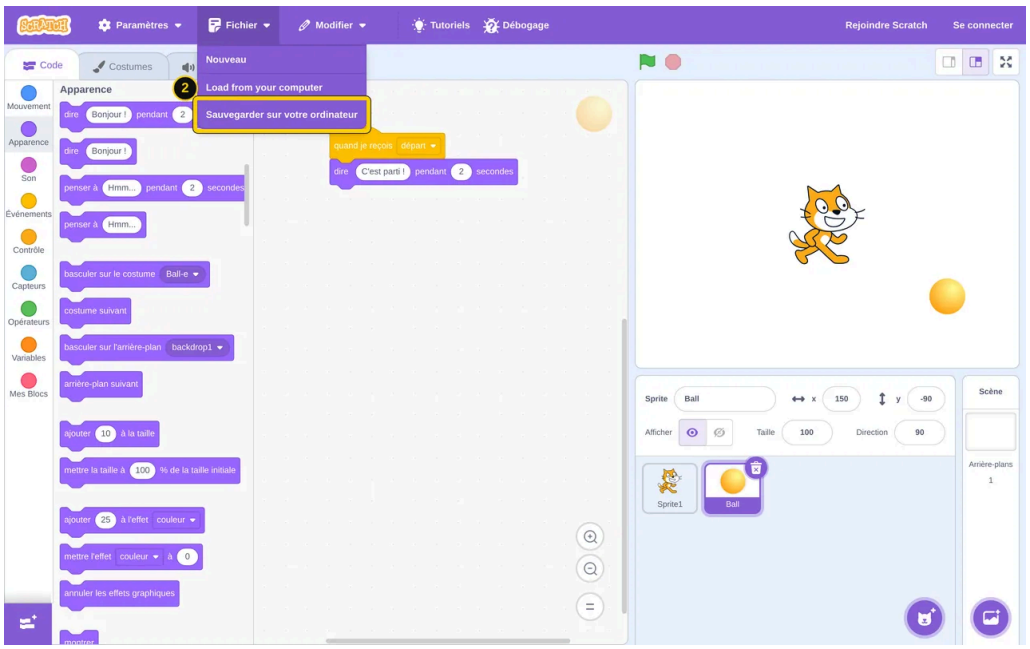
Pas à pas — enregistrer votre projet sur l'ordinateur
Étape 1. Dans la barre violette, en haut, cliquez sur **Fichier**.





Vous devez voir : un menu de trois lignes : « Nouveau », « Load from your computer » (ce libellé n'est pas traduit par Scratch : il ouvre un projet enregistré) et « Sauvegarder sur votre ordinateur ».

Étape 2. Cliquez sur **Sauvegarder sur votre ordinateur**.





Vous devez voir : le navigateur télécharge le fichier `Projet Scratch.sb3`, ou vous demande où l'enregistrer. Rangez-le dans votre dossier de formation, à un endroit que vous retrouverez.

Captures de l'éditeur Scratch, cadres compris, sous licence CC BY-SA 2.0 : « Scratch is developed by the Lifelong Kindergarten Group at the MIT Media Lab. See http://scratch.mit.edu ».

Cahier des charges

- Au moins **2 sprites**, dont au moins un autre que le chat.
- Au moins **3 scripts** au total ; au moins **1 événement** clavier ou souris.

- Au moins **1 boucle**, **1 condition** et **1 variable**.
- Au moins **1 message** : « envoyer à tous » et « quand je reçois ».
- Un titre et une consigne pour le joueur, visibles sur la scène.

Critères de réussite

- □ Toutes les exigences du cahier des charges sont présentes
- □ Le projet démarre au drapeau vert, sans erreur
- □ Un camarade a testé le projet sans votre aide (il joue, vous notez ce qui ne marche pas)
- □ Le projet est sauvegardé : le lien (si vous avez un compte et l'avez partagé) **ou** le fichier `.sb3`, déposé auprès du formateur

Si le temps le permet

- Faites tester votre projet par un deuxième camarade.
- Ajoutez un niveau de difficulté ou un meilleur score.

Pour aller plus loin

- Créez votre propre bloc : catégorie **Mes Blocs**, **Créer un bloc**.
- Ajoutez-lui une entrée (**Ajouter une entrée**) et utilisez-la.

Un bloc personnalisé est une première forme de **fonction** : un nom, des entrées, un comportement. Vous les retrouverez en JavaScript la semaine prochaine.

À retenir — cet après-midi

À retenir

- **Binaire** : tout est nombre, et chaque nombre s'écrit en 0 et 1.
- **Ordinateur** : le disque garde, la mémoire pose, le processeur exécute.
- **Message** : « envoyer à tous » et « quand je reçois », même nom.

Le détail, notion par notion :

- **Binaire** : deux chiffres, 0 et 1 ; chaque carte vaut le double de la suivante ; un texte et une image sont des nombres.
- **Dans l'ordinateur** : le disque garde les fichiers, même éteint ; la mémoire vive s'efface à l'arrêt ; le processeur exécute une instruction à la fois. Enregistrez souvent.

- **Messages** : un sprite envoie, les autres reçoivent ; le nom doit être identique ; choisissez-le dans la liste.

Sources

CS Unplugged. s. d. « Binary numbers ». Consulté le 30 septembre 2026. https://www.csunplugged.org/en/topics/binary-numbers/.

CS50. 2026a. « Lecture 0 - CS50x 2026 ». https://cs50.harvard.edu/x/notes/0/.

CS50. 2026b. « Starting from Scratch - CS50x 2026 ». https://cs50.harvard.edu/x/psets/0/scratch/.

Delacroix-Gouin, Joëlle. 2019. « Mémoire et unité centrale, un couple dédié à l'exécution des programmes ». juin 21. https://interstices.info/memoire-et-unite-centrale-un-couple-dedie-a-lexecution-des-programmes/.

Scratch Foundation. s. d.-a. « scratch-l10n : traduction française de l'interface (editor/interface/fr.json) ». Consulté le 26 septembre 2026. https://raw.githubusercontent.com/scratchfoundation/scratch-l10n/master/editor/interface/fr.json.

Scratch Foundation. s. d.-b. « scratch-l10n : traduction française des blocs (editor/blocks/fr.json) ». Consulté le 26 septembre 2026. https://raw.githubusercontent.com/scratchfoundation/scratch-l10n/master/editor/blocks/fr.json.